

1 Parte Prima: Sincronizzazione e Concorrenza

L'esecuzione concorrente di processi è una pietra miliare dei moderni sistemi operativi. Essa, tuttavia, introduce complessità legate alla necessità di coordinare l'accesso a risorse condivise per prevenire condizioni di corsa e inconsistenza dei dati. I meccanismi fondamentali per garantire questa coordinazione sono i semafori e, in modo più strutturato, i monitor.

1.1 Capitolo 1: Sincronizzazione tra Processi con Semafori e Monitor

1.1.1 1.1 Premessa sui Semafori

I semafori, introdotti da Dijkstra, sono una primitiva di sincronizzazione classica. Un uso scorretto, tuttavia, può generare errori sottili e difficili da individuare. Il pattern corretto richiede che ogni processo esegua `wait(mutex)` prima di entrare nella sezione critica e `signal(mutex)` all'uscita (p. 1).

Le violazioni di questo protocollo portano a scenari di errore specifici:

- **Inversione di ordine:** Eseguire `signal(mutex)` prima della sezione critica e `wait(mutex)` dopo di essa. Questo permette a più processi di entrare simultaneamente nella sezione critica, violando la mutua esclusione (p. 1).
- **Sostituzione di signal con wait:** Se un processo esegue `wait(mutex)`, entra nella sezione critica, e poi esegue un secondo `wait(mutex)`, si blocca permanentemente su quest'ultimo, causando un deadlock del processo stesso (p. 1).
- **Omissione:** Dimenticare una o entrambe le primitive `wait()` o `signal()` porta o a una violazione della mutua esclusione o a un blocco indefinito del processo, a seconda del contesto (p. 1).

Un aspetto cruciale è che tali difficoltà possono insorgere anche se un solo processo non si comporta correttamente, rendendo il debugging estremamente complesso (p. 1).

1.1.2 1.2 Costrutto Monitor

Per ovviare alla fragilità dei semafori, è stato introdotto il costrutto monitor (p. 2). Un monitor è un tipo di dato astratto (ADT) che incapsula i dati condivisi e fornisce un insieme di funzioni per operare su di essi. La caratteristica distintiva è che garantisce la mutua esclusione in modo implicito: al suo interno può essere attivo un solo processo alla volta (p. 2).

Struttura di un Monitor (p. 2): Un monitor è composto da:

- Dichiarazione delle variabili condivise: che definiscono lo stato dell'istanza del monitor.
- Dichiarazione del corpo delle funzioni e procedure: che operano sulle variabili e sono eseguite in mutua esclusione.
- Codice di inizializzazione: per impostare lo stato iniziale delle variabili.

Le funzioni del monitor hanno accesso solo ai loro parametri formali e alle variabili locali condivise, mentre l'esterno non può visualizzare direttamente lo stato interno del monitor. Questo incapsulamento, unito alla mutua esclusione automatica, solleva il programmatore dalla codifica esplicita dei vincoli di sincronizzazione, riducendo gli errori (p. 2).

1.2.1 Tipo Condition Per implementare schemi di sincronizzazione più complessi, come l'attesa condizionale, i monitor offrono le variabili di condizione (p. 2). Una variabile di condizione non memorizza un valore, ma è un punto di attesa su cui un processo può sospendersi. Le due primitive associate sono:

- **x.wait():** Il processo che la invoca si sospende sulla variabile x, rilasciando implicitamente il lock del monitor, permettendo ad altri processi di entrarvi.
- **x.signal():** Risveglia un singolo processo sospeso su x. Se non ci sono processi in attesa, l'operazione non ha effetto. Questa è una differenza sostanziale rispetto ai semafori, dove **signal()** modifica sempre lo stato del semaforo (p. 2).

Quando un processo P esegue **x.signal()** e un processo Q è in attesa su x, si pone il problema di chi debba continuare l'esecuzione, dato che entrambi non possono essere attivi contemporaneamente nel monitor. Due sono le strategie (p. 3):

- **Segnala e Attendi (Signal and Wait):** P attende che Q lasci il monitor o attenda su un'altra condizione.
- **Segnala e Proseguì (Signal and Continue):** Q attende che P lasci il monitor o attenda su un'altra condizione.

Alcuni linguaggi adottano un compromesso in cui il segnalante (P) lascia immediatamente il monitor, permettendo a Q di riprendere l'esecuzione.

1.2.2 Esempio di Monitor: Produttore-Consumatore (p. 3) Un classico esempio è il problema del produttore-consumatore con buffer limitato (N). Il monitor **ProducerConsumer** gestisce un buffer condiviso e le variabili di condizione **full** e **empty**.

- **Funzione enter() (Produttore):** Se il buffer è pieno (**count == N**), il produttore si blocca su **full.wait()**. In caso contrario, inserisce l'item (**put_item()**), incrementa **count** e, se il buffer era vuoto (**count == 1**), esegue **empty.signal()** per risvegliare un consumatore in attesa (p. 3-4).
- **Funzione remove() (Consumatore):** Se il buffer è vuoto (**count == 0**), il consumatore si blocca su **empty.wait()**. Altrimenti, rimuove l'item (**remove_item()**), decrementa **count** e, se il buffer era pieno (**count == N-1**), esegue **full.signal()** per risvegliare un produttore in attesa (p. 4).

L'uso del monitor rende la logica di sincronizzazione più chiara e meno soggetta a errori rispetto all'uso diretto dei semafori. Lo stesso documento mostra anche come implementare un semaforo binario utilizzando un monitor, con le funzioni **entry P()** e **V()** (p. 4).

Problemi legati ai monitor (p. 5): Nonostante i benefici, i monitor presentano due limitazioni principali:

- Sono disponibili in pochi linguaggi di programmazione.
- Richiedono la presenza di memoria condivisa, rendendoli inadatti per sistemi distribuiti o a memoria non condivisa.

1.1.3 1.3 Sincronizzazione in Java (p. 5)

Java integra il concetto di monitor a livello di linguaggio. Ogni oggetto in Java può agire come un monitor.

- **Parola chiave synchronized:** I metodi o i blocchi contrassegnati da `synchronized` possono essere eseguiti da un solo thread alla volta per oggetto. Se applicato a un metodo `static`, il lock è a livello di classe, non di oggetto (p. 5).
- **Meccanismi di attesa/notifica:** Java fornisce i metodi `wait()`, `notify()` e `notifyAll()` (ereditati da `Object`) che implementano rispettivamente wait e signal delle variabili di condizione (p. 5). L'esempio del `BoundedBuffer` in Java mostra come un produttore usi `wait()` quando il buffer è pieno e `notifyAll()` dopo aver depositato un item, e il consumatore usi `wait()` quando il buffer è vuoto e `notifyAll()` dopo aver rimosso un item. Si noti l'uso del ciclo `while` per la condizione di attesa, per gestire correttamente i "risvegli spuri" (p. 5).

1.1.4 1.4 Il Problema del Deadlock (p. 6)

Il deadlock (o stallo) è una situazione in cui un insieme di processi è bloccato in attesa di un evento che può essere generato solo da un processo dello stesso insieme. È un errore fatale che impedisce al sistema di progredire.

Un esempio comune è quello di due auto su un ponte a senso unico che si incontrano a metà; nessuna delle due può proseguire senza che l'altra arretri (p. 6). Formalmente, si verifica un deadlock quando sono soddisfatte simultaneamente le seguenti quattro condizioni (p. 7):

- **Mutua Esclusione (Mutual Exclusion):** Almeno una risorsa deve essere mantenuta in modo non condivisibile.
- **Hold and Wait (Possesso e Attesa):** Deve esistere un processo che detiene almeno una risorsa e attende di acquisirne un'altra, attualmente detenuta da un altro processo.
- **No Preemption (Nessuna Prelazione):** Le risorse non possono essere rilasciate forzatamente; devono essere rilasciate volontariamente dal processo che le detiene.
- **Attesa Circolare (Circular Wait):** Deve esistere una catena circolare di processi, dove ogni processo attende una risorsa detenuta dal successivo nella catena.

La presenza di queste condizioni non garantisce che il deadlock si verifichi, ma lo rende possibile, dipendendo dallo scheduler. La strategia di prevenzione si basa sul rendere falsa almeno una di queste quattro condizioni (p. 7).

1.1.5 1.5 Gestione dei Deadlock (p. 9)

Esistono diversi approcci per gestire i deadlock.

Prevenzione Statico: Mira a violare una delle quattro condizioni necessarie.

- **Hold and Wait:** Un processo deve richiedere tutte le risorse di cui ha bisogno all'inizio. Questo causa un basso utilizzo delle risorse e può portare a starvation (p. 9).
- **No Preemption:** Se un processo non può ottenere una risorsa, deve rilasciare tutte quelle che detiene. Fattibile solo per risorse con stato facilmente ripristinabile (CPU, registri), non per stampanti o nastri (p. 9).

- **Attesa Circolare:** Si impone un ordinamento globale per tutte le risorse. Un processo può richiedere risorse solo in ordine crescente, rendendo impossibile un ciclo di attesa (p. 9-10).

Prevenzione Dinamico (p. 10): L'allocazione delle risorse viene valutata di volta in volta. L'obiettivo è mantenere il sistema in uno stato safe, ovvero uno stato in cui esiste una sequenza safe di esecuzione dei processi che garantisce che tutti possano completare il loro lavoro.

Stato Safe: Esiste una sequenza di processi $\langle P_1, \dots, P_n \rangle$ tale che per ogni P_i , le risorse che richiede possono essere soddisfatte usando le risorse attualmente disponibili più le risorse detenute da tutti i processi P_j con $j < i$. Se questa condizione non è verificata, il sistema è in uno stato unsafe, che non è necessariamente un deadlock ma può portarvi (p. 10). L'esempio numerico sui processi P0, P1, P2 (p. 11) illustra questa transizione da stato safe a unsafe.

Per implementare la prevenzione dinamica si utilizzano:

- **Algoritmo RAG (Resource Allocation Graph) (p. 11):** Usato quando ogni tipo di risorsa ha una sola istanza. Introduce gli archi di reclamo (linea tratteggiata) che rappresentano richieste future. Una richiesta viene concessa solo se la sua trasformazione da arco di reclamo a arco di richiesta non crea un ciclo nel grafo, che indicherebbe un potenziale deadlock.
- **Algoritmo del Banchiere (Banker's Algorithm) (p. 12):** Più generale, per sistemi con più istanze per risorsa. Richiede che ogni processo dichiari il numero massimo di risorse di cui avrà bisogno. Le strutture dati sono: Disponibili (vettore), Massimo (matrice $n \times m$), Assegnate (matrice $n \times m$), Necessità (matrice $n \times m$). L'algoritmo di allocazione `request()` simula l'assegnazione delle risorse richieste e, solo se il risultato è uno stato safe (verificato dall'algoritmo `state_safe()`), concede le risorse; altrimenti, effettua un rollback e fa attendere il processo (p. 13-15). L'esempio dettagliato con 5 processi e 3 risorse (A, B, C) mostra la dinamica di richiesta e verifica della safety (p. 14-15).

Rilevazione e Ripristino (p. 15): Si permette che i deadlock si verifichino, ma si utilizzano algoritmi per rilevarli e procedure per ripristinare il sistema.

- **Rilevazione:** L'algoritmo di rilevazione è simile a quello del banchiere, ma non usa la matrice Massimo; usa invece la matrice Richieste per le richieste correnti. Cerca periodicamente un processo che possa completare l'esecuzione con le risorse disponibili e procede. Se, al termine, rimangono processi non contrassegnati come terminati, essi sono in deadlock (p. 16-17). L'esempio sul sistema con 5 processi mostra come l'algoritmo individui il deadlock dopo una nuova richiesta di P2 (p. 17).
- **Ripristino (p. 17-18):** Una volta rilevato un deadlock, le soluzioni sono:
 - **Uccisione dei processi:** Drastica e costosa. Può essere totale (uccidere tutti i processi coinvolti) o selettiva (ucciderne uno alla volta, fino a rompere il deadlock). La scelta di quale processo uccidere può basarsi su priorità, tipo di risorse allocate, tempo rimanente, ecc.
 - **Prelazione delle risorse:** Forzare il rilascio di risorse da un processo bloccato. Questo può richiedere un rollback totale (il processo ricomincia da capo) o parziale (si ripristina a un checkpoint precedente). Il rischio è lo starvation, se la prelazione viene applicata sempre agli stessi processi, e la soluzione è considerare il numero di rollback nei fattori di costo.

2 Parte Seconda: Gestione della Memoria

La gestione della memoria è un altro pilastro dei sistemi operativi. Coinvolge l'hardware (MMU, cache), il sistema operativo e il compilatore/loader. Il suo scopo è allocare efficientemente la memoria fisica, proteggere i processi l'uno dall'altro e fornire un ambiente di esecuzione flessibile.

2.1 Capitolo 2: Gestione della Memoria Centrale

2.1.1 2.1 Introduzione (p. 19)

La memoria centrale è un grande vettore di byte, ciascuno con un indirizzo. La CPU può accedere direttamente solo a dati e istruzioni presenti nella memoria centrale o nei registri. Il tempo di accesso alla RAM è molto più lento di quello ai registri, e un accesso alla memoria può bloccare il processore in attesa (p. 19).

La soluzione a questa lentezza è l'uso di una cache (p. 20), un buffer di memoria veloce posto tra CPU e memoria centrale (spesso sul chip della CPU). La gestione della cache è tipicamente hardware, trasparente al sistema operativo.

Un'altra funzione fondamentale dell'hardware di gestione della memoria è la protezione. Si usa una coppia di registri:

- **Registro Base:** Contiene l'indirizzo fisico di partenza del processo.
- **Registro Limite:** Determina la dimensione dell'intervallo di memoria ammesso (p. 20).

Ogni indirizzo generato dalla CPU in modalità utente viene confrontato con questi registri. Se l'indirizzo supera il limite, viene generata un'eccezione (trap). Solo il sistema operativo in modalità kernel può caricare questi registri tramite istruzioni privilegiate. Questo impedisce a un programma utente di accedere a memoria del SO o di altri utenti (p. 20-21).

2.1.2 2.2 Trasformazione degli Indirizzi (Binding) (p. 21-22)

Prima di essere eseguito, un programma subisce diverse fasi di trasformazione, in cui gli indirizzi cambiano forma:

- **Indirizzi Simbolici:** Usati nel codice sorgente.
- **Indirizzi Rilocabili:** Generati dal compilatore. Sono indirizzi relativi a un'origine (tipicamente 0).
- **Indirizzi Assoluti (o Fisici):** Generati dal linker/loader o dalla MMU.

Il momento in cui avviene l'associazione (binding) tra un indirizzo logico e uno fisico definisce il tipo di rilocazione:

- **Compile-Time (Statico):** L'indirizzo fisico è noto in fase di compilazione. Se cambia, il programma va ricompilato. Poco flessibile per sistemi multiprogrammati (p. 22).
- **Load-Time (Statico):** L'indirizzo fisico è noto al momento del caricamento. Il compilatore genera codice rilocabile (offset da 0) e il loader li converte in indirizzi assoluti. Se l'indirizzo di base cambia, basta ricaricare il programma (p. 22).

- **Run-Time (Dinamico):** L'associazione è ritardata fino all'esecuzione. Richiede supporto hardware (MMU). È lo schema più flessibile, usato dalla maggior parte dei SO general-purpose, poiché permette lo spostamento dei processi in memoria durante l'esecuzione (p. 22).

2.1.3 2.3 Spazi di Indirizzi Logici e Fisici (p. 22-23)

- **Indirizzo Logico (o Virtuale):** Indirizzo generato dalla CPU.
- **Indirizzo Fisico:** Indirizzo visto dall'unità di memoria.

La distinzione è valida solo con il binding run-time. La MMU (Memory Management Unit) è il dispositivo hardware che traduce gli indirizzi virtuali in fisici. Nella sua forma più semplice, usa un registro di rilocalizzazione (o base): l'indirizzo fisico è calcolato come indirizzo logico + valore del registro base (p. 23-24). Il programma utente opera su uno spazio di indirizzi logici che va da 0 a max, che viene mappato dall'MMU a uno spazio fisico che va da R a R+max.

2.1.4 2.4 Caricamento Statico vs Dinamico (p. 24-25)

- **Caricamento Statico:** Tutto il codice del programma viene caricato in memoria all'inizio. Più veloce in esecuzione, ma spreca memoria.
- **Caricamento Dinamico:** Le procedure vengono caricate solo quando vengono chiamate. Utile per codice gestione errori o routine poco usate, migliora l'utilizzo della memoria e non richiede supporto specifico dal SO (p. 24).

2.1.5 2.5 Linking Statico vs Dinamico (p. 25)

- **Linking Statico:** Le librerie vengono incluse nel file eseguibile, aumentandone le dimensioni.
- **Linking Dinamico (DLL):** Il collegamento a librerie condivise (DLL) viene posticipato al run-time. L'eseguibile contiene solo uno stub. I vantaggi sono la riduzione dello spreco di memoria (un'unica copia della libreria in RAM per tutti i processi che la usano) e la possibilità di aggiornare la libreria senza ricompilare i programmi. A differenza del caricamento dinamico, il linking dinamico richiede supporto attivo dal sistema operativo per la gestione della condivisione e della protezione (p. 25).

2.1.6 2.6 Allocazione Contigua della Memoria (p. 26)

Con l'allocazione contigua, ogni processo risiede in una singola sezione di memoria contigua. La protezione è garantita dai registri base e limite.

2.6.1 Allocazione per Partizioni Fisse (p. 27) La memoria è divisa in un numero fisso di partizioni di dimensioni predefinite (potenzialmente diverse).

- **Assegnazione:** Un processo viene allocato alla partizione più piccola che può contenerlo.
- **Una coda per partizione:** Semplice ma poco flessibile; può creare code vuote e processi in attesa nonostante partizioni libere.
- **Coda singola:** Più flessibile con politiche di assegnazione come First-Fit, Best-Fit, ecc.

Svantaggi: Frammentazione interna (spazio inutilizzato dentro la partizione), frammentazione esterna (buchi tra le partizioni) e grado di multiprogrammazione limitato (p. 27-28).

2.6.2 Allocazione per Partizioni Variabili (p. 28) Le partizioni hanno dimensione variabile, esattamente quella necessaria al processo, eliminando la frammentazione interna. Il SO tiene traccia dei “buchi” di memoria libera.

Allocazione: Quando un processo arriva, il SO cerca un buco di dimensioni sufficienti.

Politiche di scelta del buco (p. 29):

- **First-Fit:** Alloca il primo buco abbastanza grande. Veloce e buono.
- **Best-Fit:** Alloca il buco più piccolo che può contenere il processo. Cerca tutta la lista, più lento ma riduce lo spreco.
- **Worst-Fit:** Alloca il buco più grande. Generalmente il peggiore.

Svantaggi: Rimane la frammentazione esterna, l’accumulo di tanti piccoli buchi sparsi non contigui, che nel peggiore dei casi può portare a un inutilizzo di circa il 50% della memoria (regola del 50%) (p. 30). La compattazione (spostare i processi per unire i buchi) è una soluzione possibile ma costosa, e fattibile solo con rilocalizzazione dinamica (p. 30).

2.6.3 Buddy System (p. 30-31) Un compromesso tra partizioni fisse e variabili. La memoria è vista come blocchi di dimensione potenza di 2 (2^k). Quando un processo richiede memoria, si cerca il blocco più piccolo di dimensione 2^k che lo possa contenere e lo si alloca. I blocchi liberi vengono fusi (compattati) con il loro “buddy” adiacente quando entrambi sono liberi. È più efficiente della compattazione completa ma può comunque avere frammentazione interna.

2.1.7 2.7 Paginazione (Paging) (p. 32)

La paginazione risolve il problema della frammentazione esterna. Permette che lo spazio di indirizzamento fisico di un processo non sia contiguo.

2.7.1 Metodo Base (p. 32-34)

- **Memoria Fisica:** Divisa in blocchi di dimensione fissa chiamati frame.
- **Memoria Logica (Virtuale):** Divisa in blocchi di pari dimensione chiamati pagine.
- **Traduzione:** Un indirizzo logico è diviso in numero di pagina (p) e offset (d). La MMU usa p come indice in una tabella delle pagine (relativa al processo) per trovare il numero di frame (f) in cui risiede la pagina. L’indirizzo fisico è $(f * \text{dimensione_pagina}) + d$ (p. 33).

Vantaggi: Elimina la frammentazione esterna.

Svantaggi: Introduce frammentazione interna (lo spazio inutilizzato nell’ultimo frame di un processo). La dimensione media della frammentazione interna è di mezza pagina per processo, suggerendo l’uso di pagine piccole (es. 4 KB - 8 KB) (p. 34).

2.7.2 Supporto Hardware alla Paginazione (p. 36) **Registri:** La tabella delle pagine può essere memorizzata in registri dedicati (veloci, ma adatti a tabelle piccole) o in memoria principale, usando il Page-Table Base Register (PTBR) per puntare alla tabella e il Page-Table Length Register (PTLR) per la sua dimensione (p. 36).

TLB (Translation Lookaside Buffer) (p. 36): Una piccola cache associativa ad alta velocità che memorizza le traduzioni (numero di pagina -> numero di frame) usate di recente. Il suo uso è

cruciale per ridurre il tempo medio di accesso alla memoria. Un TLB hit è molto veloce (un accesso alla memoria), mentre un TLB miss richiede un accesso alla tabella delle pagine in memoria (e possibilmente un aggiornamento della TLB). Il tasso di successo (hit ratio) è una metrica fondamentale. Ad esempio, un hit ratio dell'80% può causare un rallentamento del 20%, mentre un hit ratio del 99% lo riduce all'1% (p. 38). La formula del tempo di accesso effettivo (EAT) tiene conto di questi fattori.

2.7.3 Protezione in Paginazione (p. 38) I bit di protezione nella tabella delle pagine specificano i permessi (lettura/scrittura/esecuzione) per ogni pagina. Il bit di validità (valid/invalid) indica se una pagina fa parte dello spazio di indirizzi logico del processo. Un tentativo di accesso a una pagina contrassegnata come “non valida” genera un’eccezione (page fault), evitando accessi illegali. Questo bit aiuta anche a gestire la frammentazione interna e a marcare gli indirizzi oltre il limite effettivo del programma (p. 39).

2.7.4 Pagine Condivise (p. 40) La paginazione facilita la condivisione di codice rientrante (non modificabile). Più processi possono mappare le stesse pagine fisiche di codice (ad esempio, la libreria C libc) nelle proprie tabelle delle pagine, mentre i dati sono tenuti in pagine separate. Questo risparmia enormemente memoria fisica (p. 40).

2.1.8 2.8 Struttura della Tabella delle Pagine (p. 41)

Per spazi di indirizzi molto grandi, la tabella delle pagine diventa eccessivamente grande.

- **Paginazione Gerarchica (Multi-Level Paging) (p. 41):** Suddivide la tabella delle pagine in livelli. Ad esempio, un indirizzo logico a 32 bit con pagine da 4 KB (20 bit per la pagina, 12 per l’offset) può essere diviso in un indice di tabella esterna (p1) di 10 bit e uno di tabella interna (p2) di 10 bit. Questo crea una struttura a due livelli, riducendo drasticamente la memoria necessaria per la tabella principale. Tuttavia, per architetture a 64 bit, anche 2 o 3 livelli potrebbero essere insufficienti, portando a schemi a 4 o più livelli (p. 41-42).
- **Tabelle delle Pagine Hash (Hashed Page Tables) (p. 42):** Comune in architetture a 32 e 64 bit. Il numero di pagina virtuale è l’input di una funzione hash. Ogni elemento della tabella hash punta a una lista concatenata di elementi che collidono. Ogni elemento contiene il numero di pagina virtuale, il numero di frame e un puntatore. Una variante per spazi a 64 bit è la tabella delle pagine a gruppi (Clustered Page Table), che mappa un gruppo di pagine virtuali contigue a una singola voce della tabella hash (p. 43).
- **Tabelle delle Pagine Invertite (Inverted Page Tables) (p. 43):** Invece di avere una tabella per processo, si ha una singola tabella per l’intero sistema, con un elemento per ogni frame di memoria fisica. Ogni elemento contiene l’indirizzo virtuale della pagina che vi risiede e l’ID del processo che la possiede (ASID). La ricerca è per coppia <process-id, page-number>. È più lenta (richiede una tabella hash per velocizzare la ricerca) ma occupa molta meno memoria rispetto a tabelle per processo, specialmente con molti processi.

2.1.9 2.9 Segmentazione (p. 44)

La segmentazione fornisce una visione della memoria più vicina a quella del programmatore, dividendo il programma in segmenti logici di dimensione variabile (es. codice, dati, stack).

- **Indirizzo Logico:** Coppia <numero_segmento (s), offset (d)>.

- **Tabella dei Segmenti:** Ogni elemento ha un base (indirizzo fisico di partenza) e un limite (lunghezza del segmento). Il sistema verifica che $d < \text{limite}$ prima di calcolare l'indirizzo fisico come $\text{base} + d$ (p. 45-46).

Vantaggi: Coerenza logica, supporto nativo per protezione e condivisione a livello di segmento (p. 46-47).

Svantaggi: Necessita di allocazione dinamica di memoria (con i relativi algoritmi come first-fit) e reintroduce il problema della frammentazione esterna (p. 47).

2.1.10 2.10 Segmentazione Paginata (Paged Segmentation) (p. 48)

Combina i vantaggi di entrambe le tecniche. I segmenti vengono “paginati”: ogni segmento è diviso in pagine di dimensione fissa.

La tabella dei segmenti non punta più a un singolo blocco di memoria fisica, ma a una tabella delle pagine dedicata per quel segmento.

Un indirizzo logico diventa: $\langle \text{numero_segmento}, \text{numero_pagina}, \text{offset} \rangle$.

Vantaggi: Elimina la frammentazione esterna (grazie alla paginazione) e mantiene la vista logica e i benefici di protezione/condivisione della segmentazione (p. 48).

3 Memoria Virtuale, Memoria Secondaria e File System

Tre pilastri fondamentali dell'architettura dei sistemi operativi moderni: la memoria virtuale, la gestione della memoria secondaria, e l'interfaccia del file system. Questi tre argomenti sono strettamente interconnessi, formando il substrato su cui poggia l'astrazione delle risorse di memorizzazione offerta dal sistema operativo ai programmi e agli utenti. Il filo conduttore è il concetto di astrazione: si nascondono i dettagli fisici dei dispositivi (memoria RAM limitata, dischi meccanici lenti, organizzazione dei settori) per fornire una visione logica, uniforme e più potente (spazio di indirizzamento virtuale illimitato, file come unità di memorizzazione logica, volumi e directory come organizzazione gerarchica dei dati).

3.1 Macro-Capitolo 1: Memoria Virtuale e Paginazione (Capitolo 3)

3.1.1 1.1 Concetti Fondamentali e Vantaggi della Memoria Virtuale

La memoria virtuale è una tecnica che permette l'esecuzione di processi che non risiedono completamente nella memoria fisica (RAM). Questa separazione tra la memoria logica percepita dal programma e la memoria fisica disponibile nell'hardware costituisce il cuore dell'astrazione della memoria (p. 51). I benefici principali sono molteplici e rivoluzionari per l'architettura dei sistemi:

1. **Superamento dei Limiti Fisici:** I programmi possono essere più grandi della memoria fisica totale disponibile, permettendo l'esecuzione di applicazioni di grandi dimensioni su sistemi con RAM limitata (p. 51).
2. **Astrazione e Uniformità:** La memoria virtuale astrae la memoria centrale in un vettore di memorizzazione molto grande e uniforme. Il programmatore vede uno spazio di indirizzi virtuali contiguo e vasto, senza doversi preoccupare della frammentazione o della quantità di memoria fisica. La MMU (Memory Management Unit) si occupa di tradurre gli indirizzi logici in indirizzi fisici (p. 51-52).

3. **Multiprogrammazione Aumentata:** Poiché ogni processo occupa meno memoria fisica (solo le parti effettivamente necessarie in un dato momento), è possibile eseguire molti più processi contemporaneamente. Questo aumenta l'utilizzo e la produttività della CPU, senza penalizzare i tempi di risposta o di completamento (p. 51).
4. **Riduzione dell'I/O:** Il caricamento iniziale di un programma richiede meno operazioni di I/O dal disco, poiché si caricano solo le pagine necessarie all'avvio, non l'intero eseguibile. Questo accelera l'avvio e l'esecuzione dei processi (p. 51).
5. **Condivisione Efficace:** La memoria virtuale offre un meccanismo efficiente per la creazione dei processi (tramite la `fork()`) e per la condivisione di file e memoria tra processi diversi. Librerie di sistema condivise (in sola lettura) o regioni di memoria condivisa per la comunicazione inter-processo possono essere mappate nello spazio degli indirizzi virtuali di più processi, risparmiando memoria fisica (p. 53). Le pagine possono essere condivise durante la creazione di un processo tramite `fork()`, velocizzando la generazione di nuovi processi (p. 53).
6. **Semplificazione della Programmazione:** La gestione della memoria fisica diventa trasparente, permettendo ai programmatori di concentrarsi sul problema logico, utilizzando uno spazio degli indirizzi virtuali "sparso" (con "buchi") che può espandersi dinamicamente per lo heap e lo stack (p. 52).

Definizione - Spazio degli Indirizzi Virtuali: È la collocazione logica dei processi in memoria. Un processo inizia a un certo indirizzo logico e si estende in uno spazio di memoria contiguo dal punto di vista logico, ma non necessariamente contiguo in memoria fisica. I buchi in questo spazio (aree non utilizzate tra heap e stack) costituiscono uno **spazio degli indirizzi sparso**, che può essere riempito dinamicamente tramite l'espansione di heap e stack o il linking dinamico di librerie (p. 52-53).

3.1.2 1.2 Paginazione su Richiesta (Demand Paging)

La paginazione su richiesta è la strategia di base per implementare la memoria virtuale. L'idea centrale è di non caricare in memoria l'intero eseguibile, ma di caricare le pagine nel momento esatto in cui sono richieste (p. 53). È un sistema di paginazione con avvicendamento dei processi, ma con l'aggiunta della capacità di caricare pagine dal disco solo quando necessario.

1.2.1 Concetti Fondamentali (p. 53-54) Mentre un processo è in esecuzione, alcune delle sue pagine sono in memoria fisica, altre risiedono nella memoria secondaria (di solito l'area di swap su disco). È fondamentale un supporto hardware per distinguere tra questi due casi. Questo supporto è fornito da un **bit di validità** (o bit di presenza) nella tabella delle pagine (p. 54):

- **Valido (1):** La pagina corrispondente è valida e attualmente presente in memoria fisica.
- **Non Valido (0):** La pagina non è valida (indirizzo non utilizzato) oppure è valida ma al momento risiede nella memoria secondaria.

L'Evento Page Fault: Se il processo tenta di accedere a una pagina contrassegnata come "non valida" (ma che è valida, cioè esiste su disco), si genera un'eccezione hardware chiamata **page fault** (p. 54-55). L'hardware, traducendo l'indirizzo tramite la tabella delle pagine, rileva il bit "non valido" e causa una trap che passa il controllo al sistema operativo.

1.2.2 Procedura di Gestione del Page Fault (p. 55) La procedura di servizio del page fault è un'operazione complessa che coinvolge il sistema operativo e l'hardware del disco:

1. **Validazione del Riferimento:** Il sistema operativo controlla una tabella interna per verificare se il riferimento alla pagina è valido (cioè se l'indirizzo è legittimo per il processo). Se non è valido, il processo viene terminato (errore di segmentazione). Se è valido, si procede (p. 55).
2. **Individuazione del Frame Libero:** Si cerca un frame libero nella lista dei frame liberi. Se disponibile, lo si usa. Se non ci sono frame liberi, si deve attivare un **algoritmo di sostituzione delle pagine** (descritto in dettaglio nella Sezione 1.3) per selezionare un frame "vittima" da liberare (p. 55).
3. **Avvio dell'I/O:** Il sistema operativo programma un'operazione di lettura dal disco per trasferire la pagina desiderata nel frame appena assegnato.
4. **Attesa del Completamento I/O:** Il processo rimane in attesa fino al completamento del trasferimento dal disco.
5. **Aggiornamento delle Tabelle:** Al completamento della lettura, il sistema operativo aggiorna la tabella delle pagine, contrassegnando la pagina come "valida" e presente in memoria.
6. **Riavvio dell'Istruzione:** Il sistema operativo riavvia l'istruzione che era stata interrotta dall'eccezione di page fault. A questo punto, il processo può accedere alla pagina come se fosse sempre stata in memoria (p. 55).

Caso Estremo - Pure Demand Paging: Un sistema che non carica nemmeno la prima pagina è detto a "paginazione su richiesta pura". Un processo viene avviato senza pagine in memoria. Il program counter punta alla prima istruzione, che non è in memoria, generando immediatamente un page fault. Si carica la pagina, poi si generano altri fault fino a quando tutte le pagine necessarie non sono in memoria (p. 55-56). Il meccanismo richiede che il sistema operativo possa rieseguire qualsiasi istruzione dopo un page fault, ripristinando lo stato del processo esattamente al punto di interruzione (p. 56).

1.2.3 Prestazioni della Paginazione su Richiesta (p. 56-58) L'impatto sulle prestazioni è significativo e dipende principalmente dal **tasso di page fault** (probabilità p che un accesso alla memoria generi un page fault).

Il **tempo di accesso effettivo** si calcola come:

$$\text{Tempo_Accesso_Effettivo} = (1-p) \times \text{Tempo_Accesso_Memoria} + p \times \text{Tempo_Servizio_Page_Fault}$$

- **Tempo_Accesso_Memoria (ma):** 10-200 ns.
- **Tempo_Servizio_Page_Fault:** Componente dominante. Include il tempo di trap, il salvataggio dello stato, il tempo di I/O del disco (lettura della pagina), e il ripristino. Il tempo di I/O su disco è preponderante, circa 8 ms (p. 57).

Esempio Numerico (p. 57): Considerando un tempo di accesso alla memoria di 200 ns e un tempo di servizio del page fault di 8 ms (8.000.000 ns):

$$\text{Tempo_Effettivo} = (1 - p) \times 200 + p \times 8.000.000 = 200 + 7.999.800 \times p$$

Questo mostra una dipendenza lineare e molto sensibile da p . Se $p = 1/1000$ (un fault ogni 1000 accessi), il tempo effettivo è di $8.2 \mu s$, un rallentamento di 40 volte. Per mantenere un rallentamento inferiore al 10%, si deve avere:

$$220 > 200 + 7.999.800 \times p \Rightarrow p < 0.00000025$$

Ovvero, meno di un page fault ogni 4.000.000 di accessi alla memoria. Mantenere basso il tasso di page fault è **assolutamente critico** per le prestazioni (p. 57-58).

Gestione dell'Area di Swap (p. 58): L'I/O sull'area di swap è generalmente più veloce di quello sul file system (blocchi più grandi, assenza di indirette). Il sistema può migliorare l'efficienza copiando l'intera immagine del processo nell'area di swap all'avvio e poi effettuando la paginazione da lì. In alternativa, si possono leggere le pagine dal file system inizialmente, ma scriverle nell'area di swap al momento della sostituzione. Per i file binari (eseguibili), alcune pagine possono essere lette direttamente dal file system e poi semplicemente sovrascritte (se non sono state modificate), usando il file system come backing store e riservando l'area di swap per la **memoria anonima** (stack, heap) (p. 58).

3.1.3 1.3 Sostituzione delle Pagine (Page Replacement)

Quando si verifica un page fault e la lista dei frame liberi è vuota (**over-allocation** o sovrallocazione della memoria), il sistema operativo deve scegliere una pagina attualmente in memoria da rimuovere per fare spazio a quella richiesta (p. 58-59). La decisione della pagina da sostituire è cruciale e viene affidata a un **algoritmo di sostituzione delle pagine**.

Obiettivo: Scegliere un algoritmo che minimizzi il tasso di page fault. La valutazione si fa su **successioni di riferimenti** (sequenze di numeri di pagina a cui il processo accede). Un algoritmo "perfetto" (ottimale) è irraggiungibile in pratica.

1.3.1 Algoritmo FIFO (First-In, First-Out) (p. 62-63)

- **Concetto:** Si sostituisce la pagina che è stata in memoria per il periodo più lungo. Si implementa con una coda.
- **Funzionamento:** Le pagine vengono aggiunte in coda quando vengono caricate; la pagina in testa alla coda è quella da sostituire.
- **Prestazioni:** Può essere inefficiente. La pagina più vecchia può essere attivamente usata (es. una variabile globale molto utilizzata).
- **Problema Critico - Anomalia di Belady (p. 63):** Aumentando il numero di frame di memoria disponibili, il numero di page fault **può aumentare**. Questa situazione controintuitiva è nota come **Anomalia di Belady** e dimostra la non-ottimalità dell'algoritmo.

1.3.2 Algoritmo Ottimale (OPT o MIN) (p. 64)

- **Concetto:** Sostituisce la pagina che non verrà usata per il periodo di tempo più lungo nel futuro.
- **Prestazioni:** Fornisce il tasso minimo di page fault per un dato numero di frame e non soffre dell'anomalia di Belady.

- **Implementabilità:** È **irrealizzabile in pratica**, perché richiede la conoscenza futura della successione di riferimenti. Viene usato solo come benchmark per confrontare altri algoritmi.

1.3.3 Algoritmo LRU (Least Recently Used) (p. 65-66)

- **Concetto:** Approssima l'algoritmo ottimale basandosi sul passato. Sostituisce la pagina che non è stata usata per il periodo di tempo più lungo (usata meno di recente). Si basa sul principio che "un passato recente è un buon predittore del futuro vicino".
- **Prestazioni:** È un algoritmo a stack (insieme di pagine in N frame è sottoinsieme di quelle in N+1 frame), quindi **non soffre dell'anomalia di Belady** (p. 66). Produce un tasso di page fault inferiore a FIFO.
- **Implementazione:** Richiede un supporto hardware significativo. Due tecniche principali (p. 65-66):
 - **Contatori:** A ogni voce della tabella delle pagine si associa un campo "momento di utilizzo". Un contatore hardware incrementa a ogni riferimento. A ogni accesso a una pagina, si copia il valore del contatore nel suo campo. Per la sostituzione, si cerca la pagina con il valore più piccolo. È costoso in termini di scritture in memoria per ogni accesso e richiede la gestione dell'overflow.
 - **Stack:** Si mantiene uno stack dei numeri di pagina. Ogni volta che si accede a una pagina, la si estrae dallo stack e la si mette in cima. La pagina in fondo allo stack è la LRU. È efficiente per la ricerca (nessuna ricerca), ma l'aggiornamento (estrazione da centro stack e reinserimento in cima) è costoso.

1.3.4 Algoritmi di Approssimazione a LRU (p. 66-67) Poiché l'LRU puro richiede hardware spesso non disponibile, si usano approssimazioni.

- **Bit di Riferimento (Reference Bit):** L'hardware imposta automaticamente un bit a 1 ogni volta che una pagina viene referenziata (lettura o scrittura). Il sistema operativo azzerà i bit periodicamente. Fornisce solo una informazione binaria (usata/non usata), non l'ordine (p. 66-67).
- **Algoritmo con Bit Supplementari (p. 67):** Si registrano i bit di riferimento a intervalli regolari (es. ogni 100ms) in un registro a scorrimento per ogni pagina. Esempio: un registro a 8 bit che viene shiftato a destra, inserendo il bit corrente nel bit più significativo. Una pagina con registro 00000000 non è stata usata per 8 periodi; 11111111 è stata usata in tutti. Si confrontano i valori numerici per approssimare l'ordine di utilizzo.
- **Algoritmo con Seconda Chance (o Clock) (p. 67-68):** Variante di FIFO che usa il bit di riferimento.
 - Si mantiene una coda circolare (orologio) di frame.
 - Un puntatore indica il prossimo frame da esaminare.
 - Per trovare una vittima, si esamina il frame puntato: se il bit di riferimento è 0, lo si sostituisce; se è 1, si azzerà il bit, si dà una "seconda chance", e si avanza il puntatore al frame successivo.
 - Nel caso peggiore (tutti i bit a 1), il puntatore compie un giro completo, azzerà tutti i bit e si comporta come FIFO.

1.3.5 Altri Algoritmi di Sostituzione (p. 68)

- **LFU (Least Frequently Used):** Sostituisce la pagina con il conteggio di riferimenti più basso. Può avere il problema di pagine usate intensamente all'inizio e poi mai più (rimangono in memoria).
- **MFU (Most Frequently Used):** Sostituisce la pagina con il conteggio più alto. Si basa sul principio che una pagina con conteggio basso è appena arrivata e non è ancora stata usata. Poco comuni e costosi.

3.1.4 1.4 Allocazione dei Frame (p. 69-70)

Il sistema operativo deve decidere quanti frame di memoria fisica assegnare a ciascun processo.

- **Numero Minimo di Frame:** Ogni processo deve avere un numero minimo di frame, dettato dall'architettura del processore (es. un'istruzione che fa riferimento a più parole di memoria, indirizzamento indiretto, ecc.). Non rispettare questo minimo è impossibile (p. 69).
- **Numero Massimo:** Definito dalla quantità totale di memoria fisica disponibile.
- **Algoritmi di Allocazione:**
 - **Allocazione Uniforme (Equal Allocation):** Ogni processo riceve lo stesso numero di frame (m/n). Non tiene conto delle diverse dimensioni dei processi (p. 70).
 - **Allocazione Proporzionale (Proportional Allocation):** Si assegnano frame proporzionalmente alla dimensione virtuale di ciascun processo. Sia s_i la dimensione virtuale del processo p_i ; $S = \sum s_i$ è la somma totale delle dimensioni. Il numero di frame a_i per il processo p_i è: $a_i = (s_i/S) \times m$, dove m è il totale dei frame disponibili. L'allocazione deve soddisfare il vincolo del numero minimo di frame.
 - **Allocazione Basata su Priorità:** Processi a priorità più alta ricevono più frame. Se un processo a priorità alta ha un page fault, può prendere frame da uno a priorità bassa.

3.1.5 1.5 Thrashing e Modelli di Località (p. 70-73)

Il **thrashing** è una condizione di degrado estremo delle prestazioni in cui il sistema spende più tempo a fare paginazione (I/O su disco) che a eseguire i processi. È la principale minaccia alle prestazioni della memoria virtuale.

1.5.1 Cause del Thrashing (p. 71) Il thrashing è una reazione a catena causata da un **eccessivo grado di multiprogrammazione**:

1. Il sistema operativo, vedendo un basso utilizzo della CPU, introduce un nuovo processo in memoria per aumentare la multiprogrammazione.
2. Il nuovo processo compete per i frame, riducendo i frame disponibili per gli altri processi.
3. I processi esistenti non hanno abbastanza frame per il loro **working set** (insieme di pagine usate attivamente). Iniziano a generare un alto numero di page fault.
4. Per far fronte ai fault, si devono sostituire pagine che sono ancora attive. I processi vengono messi in attesa per il dispositivo di paginazione.
5. L'utilizzo della CPU crolla, poiché tutti i processi sono bloccati in attesa di I/O.

6. Lo scheduler, vedendo il basso utilizzo della CPU, introduce **ancora più processi**, peggiorando la situazione. Si crea un circolo vizioso.

La figura a p. 71 mostra che l'utilizzo della CPU aumenta con la multiprogrammazione fino a un massimo, poi crolla drammaticamente a causa del thrashing.

1.5.2 Modello del Working Set (p. 72-73) Il modello del working set è una strategia per prevenire il thrashing, basata sul concetto di **località**.

- **Definizione di Località:** Insieme di pagine usate attivamente insieme. Durante l'esecuzione, un processo si sposta da una località all'altra (es. durante l'esecuzione di una funzione, la località è composta dal codice della funzione, dalle sue variabili locali e da un sottoinsieme di variabili globali) (p. 71).
- **Working Set (WS):** L'insieme di pagine referenziate nei più recenti Δ riferimenti alla memoria. Il parametro Δ definisce la finestra temporale (p. 72-73). Se una pagina è attiva, è nel working set. Se non viene referenziata per Δ unità di tempo, esce dal working set. Il working set è un'approssimazione della località corrente del processo.
- **Controllo del Thrashing con il Working Set:** Il sistema operativo stima il working set (WSS_i) di ogni processo. La richiesta totale di frame $D = \sum WSS_i$ è la somma delle dimensioni dei working set. Se $D > m$ (totale frame disponibili), si ha thrashing. Per prevenirlo, il sistema operativo sospende uno o più processi (li scrive su disco), liberando i loro frame e riducendo il grado di multiprogrammazione finché $D \leq m$. Questo ottimizza l'uso della CPU (p. 73).
- **Implementazione:** È difficile tracciare esattamente il working set. Si usano approssimazioni con il bit di riferimento e timer. Le pagine con bit di riferimento attivo entro un certo intervallo di tempo sono considerate nel working set (p. 73).

1.5.3 Controllo della Frequenza dei Page Fault (PFF) (p. 74) Un'altra strategia per controllare il thrashing è misurare direttamente la **frequenza dei page fault** di ciascun processo. Si definiscono un limite superiore e uno inferiore per la frequenza accettabile.

- **Se la frequenza supera il limite superiore:** Il processo ha troppi page fault, quindi gli si assegna un frame aggiuntivo.
- **Se la frequenza scende sotto il limite inferiore:** Il processo ha troppi frame, quindi gli se ne sottrae uno.

Questo approccio permette un controllo dinamico e diretto dell'allocazione dei frame per ogni processo (p. 74).

3.2 Macro-Capitolo 2: Gestione della Memoria Secondaria e Dispositivi di Storage (Capitolo 4)

Il sistema operativo deve gestire la memoria secondaria, che fornisce un'archiviazione persistente e di massa per i dati. Questo capitolo copre l'hardware di storage, la schedulazione degli accessi e la gestione a basso livello dei dispositivi.

3.2.1 2.1 Struttura dei Dispositivi di Memorizzazione

2.1.1 Nastri Magnetici (p. 75)

- **Caratteristiche:** Supporto di memorizzazione a striscia plastica ricoperta di materiale magnetizzabile.
- **Accesso:** Puramente **sequenziale**. La testina di lettura/scrittura deve posizionarsi all'inizio del dato e leggere in modo lineare. La latenza di riposizionamento può essere di decine di secondi.
- **Utilizzo:** Una volta onnipresenti, oggi sono quasi esclusivamente usati per il **backup** di sistemi e archiviazione a lungo termine (cold storage) grazie alla loro alta capacità (fino a 5 TB per nastro nel 2011) e basso costo per GB, ma sono stati ampiamente sostituiti da dischi magnetici e SSD per l'accesso online.

2.1.2 Dischi Magnetici (HDD) (p. 75-76)

- **Struttura Fisica:** Composti da uno o più **piatti** (platter) circolari in alluminio, rivestiti di materiale magnetico su entrambe le facce. Ogni superficie è divisa logicamente in **tracce** concentriche, suddivise in **settori** (la più piccola unità di trasferimento, tipicamente 512 B o 4 kB). L'insieme di tracce alla stessa posizione su tutti i piatti forma un **cilindro**. Le **testine di lettura/scrittura** sono sospese su ogni superficie e collegate a un **braccio** che le muove in blocco (p. 76).
- **Caratteristiche Prestazionali:** La velocità di rotazione (RPM - giri al minuto) è critica (5400, 7200, 10000, 15000 RPM). Il **tempo di accesso** a un blocco è composto da:
 1. **Tempo di Ricerca (Seek Time):** Tempo per spostare il braccio sul cilindro corretto.
 2. **Latenza di Rotazione:** Tempo per far ruotare il settore desiderato sotto la testina. In media, mezza rotazione.
 3. **Tempo di Trasferimento:** Tempo per trasferire i dati letti/scritti. $\text{Transfer Rate} = \text{Dimensione Blocco} / \text{Velocità di Trasferimento}$.
- **Esempio di Calcolo (p. 77):** Con Transfer Rate = 40 MB/s, Rotazione = 1000 RPM (166 RPS), blocco da 512 B (0.5 kB), e Seek Time di 5 ms: $\text{Tempo Accesso} = 5 \text{ ms} + (0.5 \text{ R}/166 \text{ RPS}) + (0.5 \text{ kB}/40 \text{ MB}) = 5 \text{ ms} + 3 \text{ ms} + 0.0000125 \text{ ms} \approx 8.0000125 \text{ ms}$

2.1.3 Dispositivi NVM (Non-Volatile Memory) o SSD (Solid-State Drives) (p. 77)

- **Caratteristiche:** Basati su memoria flash NAND. Non hanno parti mobili, risultando più affidabili, più veloci (nessun tempo di posizionamento o latenza di rotazione) e con minore consumo energetico rispetto agli HDD.
- **Svantaggi:** Costo per megabyte più elevato e capacità inferiore. Inoltre, il bus standard (es. SATA) può limitare il throughput; per questo molti SSD si connettono direttamente al bus PCIe.
- **Problematiche di Scrittura:** I semiconduttori NAND non possono essere sovrascritti senza prima essere cancellati a livello di blocco. La cancellazione è un'operazione lenta, ma mitigata dalla parallelizzazione su più "die" (chip). La maggiore limitazione è il **deterioramento**: le celle NAND hanno un numero finito di cicli di scrittura/cancellazione (circa 100.000). La

durata si misura in scritture totali, non in anni. I controllori SSD implementano algoritmi di “wear leveling” e gestione dei blocchi difettosi per mitigare questo problema, trasparenti al sistema operativo (p. 77).

3.2.2 2.2 Scheduling dei Dischi Rigidi (Disk Scheduling) (p. 77-82)

Per i dischi meccanici, il movimento del braccio e la rotazione del piatto sono costosi. L'**algoritmo di scheduling** del disco ha il compito di ordinare le richieste di I/O in sospeso per minimizzare i tempi di ricerca e di latenza, migliorando il throughput e il tempo medio di risposta.

La coda delle richieste può essere lunga. Le richieste contengono: operazione (lettura/scrittura), indirizzo di memoria, e numero di settori (p. 78).

2.2.1 FCFS (First-Come, First-Served) (p. 78)

- **Concetto:** Servire le richieste nell'ordine di arrivo.
- **Prestazioni:** Semplice ed equo, ma inefficiente. Causa movimenti lunghi e non ottimizzati della testina (es. salti da un estremo all'altro del disco). Esempio: con richieste [98, 183, 37, 122, 14, 124, 65, 67] e testina a 53, il movimento totale è di 640 cilindri (p. 78).

2.2.2 SSTF (Shortest Seek Time First) (p. 79)

- **Concetto:** Seleziona la richiesta con il minimo spostamento del braccio rispetto alla posizione corrente.
- **Prestazioni:** Migliore di FCFS (esempio: movimento 236 cilindri). È un algoritmo “greedy” e non è ottimale.
- **Problema:** Può causare **starvation** (attesa indefinita). Se arrivano continuamente richieste vicine alla posizione corrente, le richieste ai cilindri lontani potrebbero non essere mai servite.

2.2.3 SCAN (p. 79)

- **Concetto (Algoritmo dell'Ascensore):** Il braccio parte da un estremo e si muove verso l'altro, servendo le richieste incontrate. Quando raggiunge l'estremo opposto, inverte la direzione e continua.
- **Prestazioni:** Più equo di SSTF. Le richieste vengono servite in un ordine determinato dalla posizione e dalla direzione. Nell'esempio, partendo da 53 e andando verso 0, si servono 37, 14, poi si inverte e si servono 65, 67, 98, 122, 124, 183 (p. 79).

2.2.4 C-SCAN (Circular SCAN) (p. 80)

- **Concetto:** Variante di SCAN. Il braccio si muove in una sola direzione, servendo le richieste. Quando raggiunge l'estremo opposto, ritorna **senza servire** alla posizione iniziale e ricomincia. Tratta il disco come una lista circolare.
- **Prestazioni:** Fornisce un tempo di attesa più **uniforme** rispetto a SCAN, prevenendo il favoritismo per i cilindri centrali.

2.2.5 LOOK e C-LOOK (p. 80-81) Varianti di SCAN e C-SCAN che non portano il braccio fino all'estremo del disco se non ci sono richieste in quella direzione. Il braccio inverte la marcia (LOOK) o torna indietro (C-LOOK) quando non ci sono più richieste davanti. Migliorano le prestazioni riducendo i movimenti inutili.

2.2.6 Altri Algoritmi (p. 81)

- **N-step SCAN:** Divide la coda in sotto-code di dimensione massima N per evitare che un processo monopolizzi la testina. N grande $\rightarrow$ SCAN, N=1 $\rightarrow$ FCFS.
- **FSCAN:** Usa due code, una attiva e una passiva. Durante uno SCAN, le nuove richieste vanno nella coda passiva, garantendo equità.
- **LIFO (Last-In, First-Out):** A volte utile per accessi con elevata località, ma può causare starvation.

3.2.3 2.3 Gestione delle Unità di Memoria Secondaria (p. 81-84)

Prima che un dispositivo di memorizzazione possa essere usato dal sistema operativo per contenere file, deve essere preparato.

2.3.1 Formattazione e Partizioni (p. 81-83)

1. **Formattazione Fisica (Basso Livello):** Divisione del dispositivo in settori (o pagine per NVM). Ogni settore ha una struttura: intestazione (contiene info di controllo, ECC), area dati, e coda. Di solito eseguita dal produttore. La dimensione del settore è una scelta di progetto (es. 512 B, 4 kB) che bilancia overhead (intestazioni/coda) e dimensione dei dati utente (p. 81-82).
2. **Partizionamento:** Il dispositivo viene suddiviso in uno o più gruppi di blocchi (o pagine), detti **partizioni**. Ogni partizione può contenere un file system separato (es. una per il sistema operativo, una per i dati utenti). Le informazioni sulle partizioni sono in una posizione fissa (p. 82).
3. **Creazione del Volume:** A volte implicita (un file system su una partizione), a volte esplicita (RAID, volumi logici). Il termine “volume” indica un file system montabile.
4. **Formattazione Logica (Creazione del File System):** Il sistema operativo scrive le strutture dati iniziali del file system sulla partizione/volume: descrizione dello spazio libero, dello spazio allocato, e una directory iniziale vuota (p. 82). Se la partizione è **avviabile** (bootable), contiene il sistema operativo. La partizione di avvio diventa la radice (**root**) del file system, da cui si possono montare gli altri volumi.

2.3.2 Blocchi Difettosi (Bad Blocks) (p. 83) I dischi possono sviluppare settori malfunzionanti. La gestione varia:

- **Dischi Semplici (IDE):** Gestione manuale. Il sistema operativo scansiona il disco per blocchi difettosi e li marca come inutilizzabili, oppure isola i blocchi che si guastano durante l'uso tramite programmi speciali. I dati in un blocco difettoso sono persi (p. 83).
- **Dischi Avanzati (SCSI, e molti HDD/SSD moderni):** Gestione automatica (sector sparing/forwarding). Il controllore mantiene una lista di settori difettosi e una riserva di

settori buoni. Quando un settore si guasta, il controllore lo sostituisce logicamente con uno di riserva. Il sistema operativo non si accorge del guasto (vede solo il blocco logico funzionante) (p. 83).

2.3.3 Gestione dell'Area di Avvicendamento (Swap Space) (p. 84) L'area di swap è lo spazio su disco usato per la memoria virtuale. L'obiettivo è fornire il massimo throughput per il sistema di memoria virtuale. La progettazione si concentra sull'ottimizzazione dell'I/O, spesso usando un'area dedicata e non un file system per evitare overhead.

3.3 Macro-Capitolo 3: File System - Interfaccia e Organizzazione (Capitolo 5)

Il file system è la parte del sistema operativo che fornisce un meccanismo di memorizzazione persistente e condivisa dei dati. Offre una visione logica (astrazione) dei dispositivi di memorizzazione fisici sotto forma di file, organizzati in una struttura gerarchica.

3.3.1 3.1 Concetto di File (p. 85)

Un **file** è un'unità logica di memorizzazione, definita come un insieme di informazioni correlate a cui è stato assegnato un nome e che risiede su un dispositivo di memoria secondaria non volatile.

Attributi del File (p. 85-86):

- **Nome:** Nome simbolico, l'unico modo "umano" di riferirsi al file.
- **Identificatore:** Etichetta unica (es. numero) usata internamente dal sistema operativo.
- **Tipo:** Informazione per gestire file diversi (eseguibile, oggetto, sorgente, testo, ecc.).
- **Locazione:** Puntatore al dispositivo e alla posizione fisica del file.
- **Dimensione:** Dimensione corrente del file (in byte, parole o blocchi).
- **Protezione:** Informazioni di controllo (chi può leggere, scrivere, eseguire).
- **Data/Ora:** Creazione, ultima modifica, ultimo accesso.

Operazioni sui File (p. 86-87):

- **Creazione:** Cerca spazio nel file system e crea un nuovo elemento nella directory.
- **Scrittura:** Scrive dati a partire da un puntatore di scrittura, che viene aggiornato.
- **Lettura:** Legge dati a partire da un puntatore di lettura, che viene aggiornato. Spesso lo stesso puntatore è usato per lettura e scrittura.
- **Riposizionamento (Seek):** Cambia il valore del puntatore alla posizione corrente del file.
- **Cancellazione:** Cerca il file nella directory, rilascia lo spazio su disco e rimuove l'elemento dalla directory.
- **Troncamento:** Cancella il contenuto del file mantenendo gli attributi.
- **Aggiunta (Append):** Scrive nuove informazioni alla fine del file.

Apertura dei File e Tabelle dei File Aperti (p. 87): Per evitare di cercare il file nella directory a ogni operazione (costo alto), il sistema operativo mantiene una **tabella dei file aperti**. Quando un processo apre un file, viene aggiunta una voce in questa tabella. Le operazioni successive (lettura,

scrittura) si riferiscono al file tramite un indice (file descriptor) in questa tabella. Le informazioni associate a un file aperto includono: puntatore alla posizione corrente (per processo), contatore del numero di processi che lo hanno aperto (per gestire la chiusura), posizione su disco, e diritti di accesso (p. 87).

3.3.2 3.2 Metodi di Accesso ai File (p. 88)

- **Accesso Sequenziale:** Le informazioni vengono lette/scritte in ordine, un record dopo l'altro. Le operazioni principali sono `read_next()` e `write_next()`. Un file può anche permettere di avanzare/indietreggiare di n record. È il modello del nastro magnetico, ma funziona anche su dischi. È il metodo più comune (p. 88).
- **Accesso Diretto (o Casuale):** Il file è composto da record di lunghezza fissa numerati. È possibile leggere o scrivere il record n senza dover leggere quelli precedenti. È il modello del disco e permette un accesso rapido a qualsiasi parte del file. Non ci sono restrizioni sull'ordine di lettura/scrittura (p. 88).

3.3.3 3.3 Struttura delle Directory (p. 88-92)

La directory è un file speciale che organizza i file in una struttura gerarchica, fornendo un'astrazione del nome e della collocazione dei file. Le operazioni di base su una directory sono: ricerca, creazione, cancellazione, elencazione, ridenominazione e attraversamento del file system (p. 88-89).

3.3.1 Directory a un Livello (p. 89)

- **Struttura:** Un'unica directory contenente tutti i file del sistema.
- **Problemi:** Diventa ingestibile con molti file. Impone l'**unicità dei nomi** a livello globale. Problemi di organizzazione e di collisione di nomi tra utenti diversi.

3.3.2 Directory a due Livelli (p. 89-90)

- **Struttura:** Una **Directory Principale (Master File Directory, MFD)** che contiene le **Directory Utente (User File Directory, UFD)**, una per ogni utente. Ogni UFD contiene i file di quell'utente.
- **Vantaggi:** Risolve il problema dell'unicità dei nomi: utenti diversi possono avere file con lo stesso nome. Il sistema operativo cerca solo nell'UFD dell'utente corrente per le operazioni sui file, rendendo più efficienti le operazioni.
- **Svantaggi:** Isola gli utenti, rendendo difficile la condivisione di file. Richiede l'uso di **pathname** (percorsi) per accedere ai file di altri utenti (es. `C:\utenteB\prova.txt`). La sequenza di directory cercate per un riferimento è il **percorso di ricerca (search path)**.

3.3.3 Directory con Struttura ad Albero (p. 90-91)

- **Struttura:** Generalizzazione della directory a due livelli. Ogni utente può creare **sottodirectory** per organizzare i propri file, formando un albero con una **directory radice (root)**. Ogni file ha un unico **nome di percorso (pathname)**.
- **Concetti Chiave:**

- **Directory Corrente:** Ogni processo ha una directory corrente (di solito la home dell'utente). Se si fa riferimento a un file, si cerca prima in questa directory (p. 90).
- **Pathname Assoluto:** Inizia dalla radice (`/home/utente/documento.txt`).
- **Pathname Relativo:** Inizia dalla directory corrente (`documento.txt`).

3.3.4 Directory con Struttura a Grafo Aciclico (p. 91)

- **Struttura:** Permette la **condivisione** di file e sottodirectory tra utenti diversi. Un file può essere presente in due directory diverse, ma è la stessa entità fisica. Ciò si realizza tramite **collegamenti (link)**. Un link è un puntatore a un altro file (o directory).
- **Vantaggio:** Permette un accesso condiviso senza duplicare i dati fisici.
- **Problema:** Eliminazione di un file condiviso. Si usa un **contatore di riferimenti**. Un file viene cancellato fisicamente solo quando il contatore scende a zero.

3.3.5 Directory con Struttura a Grafo Generale (p. 92)

- **Struttura:** Permette cicli (un file o directory può essere raggiungibile da più percorsi anche attraverso un loop). I cicli complicano notevolmente gli algoritmi di attraversamento e di cancellazione.
- **Problema:** Il contatore di riferimenti non è più sufficiente per stabilire quando un file non è più referenziato (si può avere un contatore non nullo anche se non ci sono più riferimenti). Si richiede un **garbage collection** per recuperare lo spazio, attraversando l'intero grafo per contrassegnare i file accessibili e recuperare quelli non contrassegnati.

3.3.4 3.4 Protezione (p. 92-93)

La protezione dei file è essenziale per garantire l'affidabilità e la sicurezza delle informazioni.

- **Affidabilità:** Protezione da danni fisici (hardware, guasti). Si ottiene attraverso il **backup** (copie multiple su nastri, dischi, ecc.).
- **Protezione (da accessi impropri):** Controllo degli accessi ai file.

Tipi di Accesso (p. 92-93): Lettura (`read`), Scrittura (`write`), Esecuzione (`execute`), Aggiunta (`append`), Cancellazione (`delete`), Elencazione (`list`).

Liste di Controllo degli Accessi (ACL - Access Control List) (p. 93):

- Associata a ogni file e directory.
- È una lista che specifica, per ogni utente (o gruppo), i tipi di accesso consentiti (lettura, scrittura, esecuzione, ecc.).
- Quando un utente tenta un'operazione su un file, il sistema operativo consulta la ACL del file.
- Se l'utente è nella lista con i permessi richiesti, l'operazione è autorizzata; altrimenti, si ha una violazione di protezione (accesso negato).

4 LABORATORIO: Segnali e Comunicazione tra Processi (IPC) in Unix/Linux

4.1 1. Segnali (Capitolo 8)

4.1.1 8.1 Concetti base (p. 1)

I segnali sono un meccanismo utilizzato dal kernel per notificare a un processo il verificarsi di eventi specifici. Possono essere generati da:

- **Errori del programma:** divisione per zero, accesso a memoria non valida.
- **Eventi di sistema:** terminazione di un processo figlio, scadenza di un timer, operazioni I/O non eseguibili.
- **Richieste dell'utente:** terminazione del programma (es. Ctrl-C).
- **Invio esplicito:** da parte di un altro processo o del processo stesso.

Quando un segnale viene ricevuto, il processo interrompe il suo flusso normale e:

1. Esegue un'**azione predefinita** (default), oppure
2. Esegue una **funzione di gestione personalizzata** (signal handler), se è stata installata. Questa operazione si chiama **intercettare il segnale** (catching).

8.1.1 Semantiche del funzionamento dei segnali (p. 1-2) Esistono due tipi di comportamento, detti *semantiche*:

- **Semantica inaffidabile (primi Unix) (p. 1):**
 - Il gestore, una volta eseguito, **non rimane attivo**. L'utente deve reinstallarlo all'interno del gestore stesso per mantenerlo attivo.
 - Possibile **perdita di segnali**: se un secondo segnale arriva mentre il gestore del primo è in esecuzione e prima della sua reinstallazione, il secondo segnale verrà gestito con l'azione predefinita, che potrebbe terminare il processo. L'azione del gestore non verrebbe eseguita.
 - Non esiste l'atomicità tra ricezione e reinstallazione, quindi ci sono **race condition**.
 - Non si possono **bloccare i segnali**.
- **Semantica affidabile (Linux e Unix moderni) (p. 2):**
 - Il gestore, una volta installato, **resta attivo**.
 - I problemi della semantica inaffidabile sono risolti.
 - Un processo può **bloccare la consegna** dei segnali.
 - Un segnale si dice: **Generato** quando il kernel lo imposta per il processo; **Consegnato** quando viene eseguita l'azione prevista; **Pendente** nel tempo che intercorre tra la generazione e la consegna. Se bloccato, il segnale resta pendente finché non viene sbloccato.

4.1.2 8.2 Tipi di segnali (p. 2)

Gli eventi che generano segnali si dividono in tre categorie:

- **Errori:** il programma ha commesso un'azione illegale. Non tutti gli errori generano segnali; spesso vengono gestiti tramite codici di ritorno delle funzioni di libreria.
- **Eventi esterni:** riguardano operazioni su file, interazioni con dispositivi o altri processi.
- **Richieste esplicite:** chiamate a funzioni di sistema come `kill()`.

I segnali si dividono ulteriormente in:

- **Sincroni:** legati a un'azione specifica del programma. Molti errori generano segnali sincroni, ma alcuni (es. divisione per zero) possono non esserlo completamente.
- **Asincroni:** generati da eventi esterni al controllo del processo, arrivano in tempi imprevedibili (es. terminazione di un processo figlio).

4.1.3 8.3 Classificazione dei segnali (p. 3-5)

Ogni segnale ha un numero, ma è sconsigliato usarlo nei programmi perché può variare. Si usano i nomi (`SIGname`) e le costanti definite in `<signal.h>`.

Tabella 8.1 (p. 3): Lista dei segnali (non completa) in Linux

Segnale	Standard	Azione	Cattura?	Descrizione
SIGTERM	PT	T	Sì	Segnale di terminazione.
SIGINT	PAT	T	Sì	Interrupt da tastiera (Ctrl-C).
SIGQUIT	PCS	C	Sì	Quit da tastiera (Ctrl-\\). Crea core dump.
SIGKILL	PT	T	No	Segnale di terminazione forzata. Non può essere catturato, ignorato o bloccato.
SIGALRM	PT	T	Sì	Segnale del timer da <code>alarm()</code> .
SIGCONT	P	S	Sì	Continua se fermato.
SIGSTOP	PS	S	No	Ferma il processo. Non può essere catturato, ignorato o bloccato.
SIGPIPE	PT	T	Sì	Pipe spezzata.
SIGUSR1	PT	T	Sì	Segnale utente numero 1.
SIGUSR2	PT	T	Sì	Segnale utente numero 2.

Legenda Tabella 8.1 (p. 3-4):

- **Standard:** P = POSIX, T = POSIX.1, A = ANSI C, S = SUSv2, V = System V.
- **Azione Predefinita (Tabella 8.3, p. 4):** T = Terminare il processo. C = Terminare il processo e scrivere un **core dump**. I = Ignorare il segnale. S = Fermare il processo (stop).

8.3.1 Segnali di terminazione (p. 3-4)

- **SIGTERM (Terminate) (p. 4):** Segnale generico per terminare un programma. Generato di default da `kill`. Può essere intercettato, ignorato o bloccato. Usato per chiedere la chiusura “formale” di un processo.
- **SIGINT (Interrupt) (p. 4):** Segnale di interruzione generato da Ctrl-C.
- **SIGQUIT (Quit) (p. 4):** Simile a **SIGINT** ma generato da Ctrl-\. L’azione predefinita termina il processo e crea un **core dump**. Quindi, se intercettato, non è opportuno fare operazioni di pulizia che cancellerebbero informazioni utili nel core dump.
- **SIGKILL (Kill) (p. 4):** **Non può essere intercettato, ignorato o bloccato.** Causa comunque la terminazione del processo. Usato come ultima risorsa quando **SIGTERM** non funziona. Può essere generato dal sistema stesso in condizioni particolari.

8.3.2 Segnali di allarme (p. 4) SIGALRM (Alarm) (p. 4): Generato alla scadenza di un timer (es. dalla funzione `alarm()`). L’azione predefinita è terminare, ma il suo uso presuppone un gestore per eseguire azioni programmate.

8.3.3 Segnali per il controllo di sessione (p. 4-5)

- **SIGCONT (Continue) (p. 4-5):** Usato per far ripartire un programma fermato da **SIGSTOP**. **Non può essere bloccato.** Causa sempre il riavvio del processo, anche se viene installato un gestore.
- **SIGSTOP (Stop) (p. 5):** Ferma l’esecuzione del processo (stato `stopped`). **Non può essere intercettato, ignorato o bloccato.**

8.3.4 Segnali di operazioni errate (p. 5) SIGPIPE (Broken Pipe) (p. 5): Generato quando si scrive su una pipe, FIFO o socket che non ha un processo in lettura. L’azione predefinita è terminare il processo. Se il segnale è bloccato, intercettato o ignorato, la chiamata che lo ha causato fallisce con errore `EPIPE`.

8.3.5 Ulteriori segnali (p. 5) SIGUSR1 e SIGUSR2 (User signals) (p. 5): Segnali a disposizione dell’utente. Generati solo tramite `kill()`. Utili per implementare comunicazione elementare tra processi o per eseguire operazioni su richiesta. Azione predefinita: terminare il processo.

4.1.4 8.4 Gestore del segnale (Signal Handler) (p. 5-8)

Un signal handler è una funzione che svolge un’azione specifica quando un segnale viene consegnato. Ha la seguente forma generale:

```
void sigHandler(int sig) {
    /* Codice del gestore */
}
```

- Il tipo di ritorno è `void`.
- L’argomento `sig` è il numero del segnale consegnato. È utile quando lo stesso gestore cattura più tipi di segnali.
- Il kernel interrompe il flusso del programma principale per eseguire il handler; al suo ritorno, riprende da dove era stato interrotto.

8.4.1 Installazione di un gestore (p. 6-7) La funzione `signal()` (definita in ANSI C) installa un gestore, ma ha limitazioni legate ai sistemi non multitasking. Il suo prototipo è:

```
#include <signal.h>
```

```
sighandler_t signal(int signum, sighandler_t handler);
```

- **Ritorna:** il gestore precedente in caso di successo, `SIG_ERR` in caso di errore.
- **errno in caso di errore:** `EINVAL`: il numero di segnale `signum` non è valido.
- **signum:** il numero del segnale (es. `SIGINT`).
- **handler:** l'indirizzo della funzione di gestione (o i valori costanti `SIG_IGN` e `SIG_DFL`).
- **sighandler_t:** è un tipo definito in glibc per semplificare:

```
typedef void (*sighandler_t)(int);
```
- **Valori speciali per handler:** `SIG_IGN`: indica che il segnale deve essere ignorato. `SIG_DFL`: ripristina l'azione predefinita.
- **Nota:** `SIGKILL` e `SIGSTOP` non possono essere ignorati né intercettati; `signal()` non ha effetto su di essi.
- La funzione restituisce l'azione precedente, che può essere salvata per essere ripristinata in seguito.

Esempio (p. 7):

```
void sigHandler(int sig) {
    printf("The signal %s was caught!\n",
        (sig == SIGINT) ? "Ctrl-C" : "signal User-1");
}

int main (int argc, char *argv[]) {
    // Si imposta sigHandler per essere eseguito con SIGINT o SIGUSR1
    if (signal(SIGINT, sigHandler) == SIG_ERR ||
        signal(SIGUSR1, sigHandler) == SIG_ERR) {
        errExit("change signal handler failed");
    }

    // ... il programma principale qui ...

    // Reimposta la disposizione di processo predefinita per SIGINT e SIGUSR1
    if (signal(SIGINT, SIG_DFL) == SIG_ERR ||
        signal(SIGUSR1, SIG_DFL) == SIG_ERR) {
        errExit("reset signal handler failed");
    }
    return 0;
}
```

Comportamento del signal handler (p. 7):

- Il segnale è asincrono, non si può prevedere il suo arrivo.

- Quando il handler viene invocato, il segnale che lo ha causato viene **automaticamente bloccato**. Viene sbloccato solo al ritorno del handler.
- Se un segnale bloccato viene generato più volte, al momento dello sblocco viene consegnato una sola volta.
- L'esecuzione del handler può essere interrotta da un altro segnale non bloccato.
- Le disposizioni del segnale vengono ereditate dal processo figlio.

8.4.2 Pausa e attesa di un segnale (p. 7-8) pause() (p. 8): Mette il processo in attesa (sleep) fino all'arrivo di un segnale.

```
#include <unistd.h>
int pause(void);
```

Ritorna sempre -1 con `errno = EINTR` (il successo è attendere indefinitamente). Usata per attendere eventi non sotto il controllo diretto del processo.

sleep() (p. 8): Mette il processo in attesa per un numero specificato di secondi.

```
#include <unistd.h>
unsigned int sleep(unsigned int seconds);
```

Ritorna 0 se l'attesa è completata, o il numero di secondi rimanenti se interrotta da un segnale. Può entrare in conflitto con `SIGALRM` su alcune implementazioni.

Esempio di pause() e sleep() (p. 8):

```
void sigHandler(int sig) {
    printf("Well done!\n");
}

int main (int argc, char *argv[]) {
    if (signal(SIGINT, sigHandler) == SIG_ERR) {
        errExit("change signal handler failed");
    }

    int time = 30;
    printf("We can wait for %d seconds!\n", time);

    // Il processo viene sospeso per massimo 30 secondi
    time = sleep(time);
    printf("%s!\n", (time == 0) ? "out of time" : "just in time");
}
```

4.1.5 8.5 Invio di un segnale (p. 9)

`kill()` - invia un segnale a uno o più processi.

```
#include <sys/types.h>
#include <signal.h>
int kill(pid_t pid, int sig);
```

- **Ritorna:** 0 in caso di successo, -1 in caso di errore.
- **errno in caso di errore:** EINVAL: segnale non valido. EPERM: permessi insufficienti. ESRCH: processo o gruppo di processi non esiste.
- **Valore di pid:** > 0: invia il segnale al processo con PID uguale a `pid`. = 0: invia il segnale a ogni processo nel process group del chiamante. = -1: invia il segnale a ogni processo (eccetto `init`). < -1: invia il segnale a ogni processo nel process group con PGID uguale a `|pid|`.
- **Segnale nullo (sig = 0):** non invia nessun segnale, ma esegue i controlli di errore. Utile per verificare l'esistenza di un processo.

Esempio di `kill()` a un processo figlio (p. 9):

```
int main (int argc, char *argv[]) {
    pid_t child = fork();
    switch(child) {
        case -1:
            errExit("fork");
        case 0: /* Processo figlio */
            while(1); // Il figlio e' bloccato in questo punto
        default: /* Processo padre */
            sleep(10); // Attende 10 secondi
            kill(child, SIGKILL); // Uccide il processo figlio
    }
    return 0;
}
```

8.5.1 Funzioni di allarme e timer (p. 10) `alarm()` - predispone l'invio di un segnale SIGALRM dopo un certo numero di secondi.

```
#include <unistd.h>
unsigned int alarm(unsigned int seconds);
```

- **Ritorna:** il numero di secondi rimanenti a un precedente allarme, o 0 se non c'era un allarme pendente.
- **seconds = 0:** cancella ogni allarme precedente.
- **Limitazioni:** Usa il clock-time (tempo reale). Precisione limitata al secondo. Genera il segnale una sola volta.

Esempio con `alarm()` (p. 10):

```
void sigHandler(int sig) {
    printf("Out of time!\n");
    _exit(0);
}

int main (int argc, char *argv[]) {
    if (signal(SIGALRM, sigHandler) == SIG_ERR) {
        errExit("change signal handler failed");
    }
}
```

```

int time = 30;
printf("We have %d seconds to complete the job!\n", time);
alarm(time); // Impostazione del timer

time = alarm(0); // Disabilitazione del timer
printf("%s seconds before timer expiration!\n", time);
return 0;
}

```

4.1.6 8.6 Impostazione e blocco di un segnale (p. 10-13)

Per superare i limiti della semantica inaffidabile (mancanza di blocco e controllo dei segnali pendenti), POSIX.1 introduce il tipo di dato `sigset_t` per rappresentare insiemi di segnali e funzioni per manipolarli.

8.6.1 Insieme di segnali (p. 11) Funzioni per la manipolazione di `sigset_t`:

```

#include <signal.h>
int sigemptyset(sigset_t *set); // Inizializza un insieme vuoto
int sigfillset(sigset_t *set); // Inizializza un insieme pieno
int sigaddset(sigset_t *set, int signum); // Aggiunge un segnale
int sigdelset(sigset_t *set, int signum); // Rimuove un segnale
int sigismember(const sigset_t *set, int signum); // Verifica appartenenza

```

Tutte ritornano 0 in caso di successo, -1 in caso di errore (EINVAL per segnale non valido). `sigismember` ritorna 1 se il segnale è nell'insieme, 0 altrimenti.

8.6.2 Blocco dell'emissione del segnale (p. 11-13) Maschera dei segnali (Signal Mask): insieme dei segnali la cui consegna è bloccata.

`sigprocmask()` - modifica la maschera dei segnali del processo corrente.

```

#include <signal.h>
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);

```

- **Ritorna:** 0 in caso di successo, -1 in caso di errore (EFAULT, EINVAL).
- **how:** specifica come modificare la maschera. `SIG_BLOCK`: Aggiunge i segnali in `set` alla maschera corrente (unione). `SIG_UNBLOCK`: Rimuove i segnali in `set` dalla maschera corrente. `SIG_SETMASK`: Imposta la maschera al valore specificato da `set`.
- **set:** insieme di segnali da usare per la modifica.
- **oldset:** (opzionale) se non NULL, vi viene salvata la maschera precedente.

Esempio: Bloccare tutti i segnali tranne SIGTERM (p. 12-13)

```

int main (int argc, char *argv[]) {
    sigset_t mySet, prevSet;

    // Inizializza mySet per contenere tutti i segnali
    sigfillset(&mySet);

```

```

// Rimuove SIGTERM da mySet
sigdelset(&mySet, SIGTERM);

// Blocca tutti i segnali a eccezione di SIGTERM
sigprocmask(SIG_SETMASK, &mySet, &prevSet);

/* Il processo non puo' essere interrotto dai segnali, tranne da SIGTERM */

// Reimposta la maschera dei segnali del processo
sigprocmask(SIG_SETMASK, &prevSet, NULL);

// Il processo non e' interrotto dai segnali in prevSet
return 0;
}

```

4.2 2. System V IPC (Capitolo 9)

4.2.1 9.1 Introduzione a System V-IPC (p. 15-17)

System V IPC (SysV-IPC) è un insieme di meccanismi di comunicazione basati su **oggetti permanenti nel kernel** (semafori, memoria condivisa, code di messaggi).

Caratteristiche:

- Non mantengono un contatore dei riferimenti (a differenza dei file descriptor).
- Devono essere cancellati esplicitamente, altrimenti restano attivi fino al riavvio del sistema.
- Possono essere cancellati anche se in uso.

Identificatori e Chiavi (p. 15-16):

- Ogni oggetto ha un **identificatore** (int) dinamico, restituito dalle funzioni di creazione.
- Per permettere a processi diversi di accedere allo stesso oggetto, si usa una **chiave** (key_t), specificata in fase di creazione.
- Struttura `ipc_perm` associata a ogni oggetto:

```

struct ipc_perm {
    key_t key;           // Chiave
    uid_t uid;           // Proprietario user ID
    gid_t gid;           // Proprietario group ID
    uid_t cuid;          // Creatore user ID
    gid_t cgid;          // Creatore group ID
    unsigned short int mode; // Permessi di Read/Write
    unsigned short int seq; // Numero di sequenza
};

```

- `ftok()` (p. 16-17): genera una chiave a partire da un file e un numero di progetto.

```

#include <sys/types.h>
#include <sys/ipc.h>

```

```
key_t ftok(const char *pathname, int proj_id);
```

Ritorna la chiave in caso di successo, -1 in caso di errore. Usa il numero di inode e il dispositivo del file per generare la chiave. Non garantisce l'univocità assoluta.

Esempio ftok() (p. 17):

```
key_t key = ftok("/mydir/myfile", 'a');
if (key == -1) {
    errExit("ftok failed");
}
int id = semget(key, 10, S_IRUSR | S_IWUSR);
if (id == -1) {
    errExit("semget failed");
}
```

9.1.1 Comandi IPC (p. 17-18)

- **ipcs**: visualizza informazioni sugli oggetti IPC nel sistema.
- **ipcrm**: rimuove oggetti IPC dal sistema. **ipcrm -Q <chiave>** o **ipcrm -q <id>** per code di messaggi. **ipcrm -M <chiave>** o **ipcrm -m <id>** per memoria condivisa. **ipcrm -S <chiave>** o **ipcrm -s <id>** per semafori.

4.2.2 9.2 Semafori (p. 18-26)

I semafori non scambiano dati, ma servono per **sincronizzare** l'accesso a risorse condivise. Sono contatori mantenuti nel kernel.

9.2.1 Creazione di un insieme di semafori (p. 18-20) **semget()** - crea o ottiene l'identificatore di un insieme di semafori.

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
int semget(key_t key, int nsems, int flag);
```

- **Ritorna**: l'identificatore (intero positivo) in caso di successo, -1 in caso di errore.
- **errno in caso di errore**: ENOSPC, EINVAL, ENOMEM.
- **nsems**: numero di semafori nell'insieme (deve essere 0 se si accede a un insieme esistente).
- **flag**: maschera di bit per i permessi e opzioni (IPC_CREAT, IPC_EXCL, IPC_PRIVATE).
- **Struttura semid_ds**: associata a ogni insieme.
- **Struttura sem**: per ogni singolo semaforo (non accessibile direttamente dallo user space).

```
struct sem {
    short sempid; // pid dell'ultima operazione
    ushort semval; // Valore corrente
    ushort semnct; // Processi in attesa di incremento
    ushort semznc; // Processi in attesa di semval = 0
};
```

```
};
```

Esempio di semget() (p. 19):

```
int semid;
key_t key = //... (si genera una chiave in qualche modo)

// A) Delega la ricerca di una chiave univoca al kernel
semid = semget(IPC_PRIVATE, 10, S_IRUSR | S_IWUSR);

// B) Crea un insieme di semafori con l'identificatore key,
//     solo se non e' gia' esistente
semid = semget(key, 10, IPC_CREAT | S_IRUSR | S_IWUSR);

// C) Crea un insieme di semafori con l'identificatore key,
//     tuttavia fallisce perche' gia' esistente
semid = semget(key, 10, IPC_CREAT | IPC_EXCL | S_IRUSR | S_IWUSR);
```

9.2.2 Operazioni di controllo dei semafori (p. 20-23) `semctl()` - esegue operazioni di controllo su un semaforo o un insieme.

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
int semctl(int semid, int semnum, int cmd);
int semctl(int semid, int semnum, int cmd, union semun arg);
```

- **Ritorna:** in caso di successo, un valore positivo (con 3 argomenti) o 0 (con 4 argomenti), -1 in caso di errore.
- **errno in caso di errore:** EACCES, EIDRM, EPERM, ERANGE.
- **semid:** identificatore dell'insieme.
- **semnum:** indice del semaforo all'interno dell'insieme.
- **cmd:** azione da eseguire (IPC_STAT, IPC_RMID, IPC_SET, GETALL, GETVAL, SETALL, SETVAL, GETNCNT, GETPID, GETZCNT).
- **arg:** unione `semun` (diversa a seconda del comando).

```
union semun {
    int val;                // per SETVAL
    struct semid_ds *buf;   // per IPC_STAT e IPC_SET
    unsigned short *array;  // per GETALL e SETALL
};
```

- **Definizione di semun:** deve essere definita nel programma.

Esempio: Cambiare permessi di un insieme di semafori (p. 22):

```
key_t key = //... (si genera una chiave)

// Creazione di un insieme di 10 semafori
```

```

int semid = semget(key, 10, IPC_CREAT | S_IRUSR | S_IWUSR);

// Si istanzia la struttura semid_ds
struct semid_ds ds;
// Si istanzia l'unione semun
union semun arg;
arg.buf = &ds;

// Si ottiene una copia della struttura semid_ds appartenente al kernel
if (semctl(semid, 0 /*ignored*/, IPC_STAT, arg) == -1) {
    errExit("semctl IPC_STAT failed");
}

// Si caricano i permessi di lettura al gruppo
arg.buf->sem_perm.mode |= S_IRGRP;

// Si carica la struttura semid_ds del kernel
if (semctl(semid, 0 /*ignored*/, IPC_SET, arg) == -1) {
    errExit("semctl IPC_SET failed");
}

```

Esempio: Rimozione di un insieme di semafori (p. 23):

```

if (semctl(semid, 0/*ignored*/, IPC_RMID, 0/*ignored*/) == -1) {
    errExit("semctl failed");
} else {
    printf("semaphore set removed successfully\n");
}

```

Esempio: Inizializzazione di un semaforo specifico (p. 23):

```

key_t key = //... (si genera una chiave)
// Si crea un insieme di 10 semafori
int semid = semget(key, 10, IPC_CREAT | S_IRUSR | S_IWUSR);

// Si imposta il valore del semaforo a 0
union semun arg;
arg.val = 0;

// Si inizializza il quinto semaforo a 0
if (semctl(semid, 5, SETVAL, arg) == -1) {
    errExit("semctl SETVAL");
}

```

Esempio: Ottenere lo stato di un insieme (p. 23):

```

key_t key = //... (si genera una chiave)
// Si crea un insieme di 10 semafori
int semid = semget(key, 10, IPC_CREAT | S_IRUSR | S_IWUSR);

// Si dichiara un vettore per memorizzare i valori dei semafori

```



```

int values[10];
union semun arg;
arg.array = values;

// Si ottiene lo stato corrente dell'insieme di semafori
if (semctl(semid, 0/*ignored*/, GETALL, arg) == -1) {
    errExit("semctl GETALL");
}

```

Esempio: Ottenere info su un singolo semaforo (p. 23):

```

key_t key = //... (si genera una chiave)
// Si crea un insieme di 10 semafori
int semid = semget(key, 10, IPC_CREAT | S_IRUSR | S_IWUSR);

// Si ottengono informazioni riguardo il primo semaforo
printf("Sem:%d getpid:%d getncont:%d getzcnt:%d\n",
    semid,
    semctl(semid, 0, GETPID, NULL),
    semctl(semid, 0, GETNCNT, NULL),
    semctl(semid, 0, GETZCNT, NULL));

```

9.2.3 Operazioni di acquisizione/rilascio dei semafori (p. 24-26) `semop()` - esegue operazioni di acquisizione o rilascio su un insieme di semafori.

```

#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
int semop(int semid, struct sembuf *sops, unsigned nsops);

```

- **Ritorna:** 0 in caso di successo, -1 in caso di errore.
- **errno in caso di errore:** E2BIG, EACCES, EAGAIN, EFBIG, EIDRM, EINTR, ENOMEM, ERANGE.
- **semid:** identificatore dell'insieme.
- **sops:** puntatore a un vettore di strutture `sembuf` (operazioni).
- **nsops:** numero di operazioni nel vettore.
- **Struttura sembuf:**

```

struct sembuf {
    unsigned short int sem_num; // Numero del semaforo
    short int sem_op;           // Operazione del semaforo
    short int sem_flg;          // Flag dell'operazione
};

```
- **sem_flg:** può essere `IPC_NOWAIT` (non bloccare) o `SEM_UNDO` (ripristina il valore all'uscita del processo).
- **sem_op:** controlla l'operazione. `sem_op > 0`: Aggiunge il valore a `semval`. Non blocca mai (permessi di scrittura). `sem_op = 0`: Attende che `semval` sia 0 (permessi di lettura). `sem_op < 0`: Attende che `semval` sia $\geq |sem_op|$, poi sottrae (permessi di scrittura).

Esempio: Inizializzazione di un vettore di sembuf (p. 26):

```
struct sembuf sops[3];

sops[0].sem_num = 0;
sops[0].sem_op = -1;    // Si sottrae 1 dal semaforo 0
sops[0].sem_flg = 0;

sops[1].sem_num = 1;
sops[1].sem_op = 2;     // Si aggiunge 2 al semaforo 1
sops[1].sem_flg = 0;

sops[2].sem_num = 2;
sops[2].sem_op = 0;     // Si attende che il semaforo 2 sia uguale a 0
sops[2].sem_flg = IPC_NOWAIT; // Non bloccare se l'operazione non puo' essere eseguita subito
```

Esempio: Esecuzione di operazioni su un insieme (p. 26):

```
struct sembuf sops[3];
// ... inizializzazione come sopra ...
if (semop(semid, sops, 3) == -1) {
    if (errno == EAGAIN) { // Il semaforo 2 sarebbe bloccato
        printf("Operation would have blocked\n");
    } else {
        errExit("semop");
    }
}
```

4.2.3 9.3 Memoria condivisa (p. 26-32)

La memoria condivisa è il meccanismo IPC più veloce: un segmento di memoria fisica viene mappato nello spazio di indirizzi virtuale di più processi. Non richiede intervento del kernel dopo il mapping. Deve essere usata con un meccanismo di sincronizzazione (come i semafori) per evitare accessi concorrenti.

9.3.1 Creare un segmento di memoria condivisa (p. 26-28) `shmget()` - crea o ottiene un segmento di memoria condivisa.

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
int shmget(key_t key, int size, int flag);
```

- **Ritorna:** l'identificatore (intero positivo) in caso di successo, -1 in caso di errore.
- **errno in caso di errore:** ENOSPC, EINVAL, ENOMEM.
- **key:** chiave associata (o IPC_PRIVATE).
- **size:** dimensione del segmento in byte (arrotondata al multiplo di PAGE_SIZE).
- **flag:** permessi e opzioni (IPC_CREAT, IPC_EXCL).

- **Struttura `shmid_ds`:** associata a ogni segmento.

```
struct shmid_ds {
    struct ipc_perm shm_perm; // Permessi operativi
    int shm_segsz;           // Dimensione del segmento (byte)
    time_t shm_atime;        // Ultimo collegamento
    time_t shm_dtime;        // Ultimo scollegamento
    time_t shm_ctime;        // Ultima variazione
    unsigned short shm_cpid; // PID del creatore
    unsigned short shm_lpid; // PID dell'ultimo operatore
    short shm_nattch;        // Numero di processi agganciati al segmento
};
```

Esempio di `shmget()` (p. 28):

```
int shmid;
key_t key = //... (si genera una chiave)
size_t size = //... (si computa la dimensione)

// A) Delega la ricerca di una chiave univoca al kernel
shmid = shmget(IPC_PRIVATE, size, S_IRUSR | S_IWUSR);

// B) Crea un segmento di memoria condivisa con l'identificatore key,
//     solo se non e' gia' esistente
shmid = shmget(key, size, IPC_CREAT | S_IRUSR | S_IWUSR);

// C) Crea un segmento di memoria condivisa con l'identificatore key,
//     ma fallisce perche' gia' esistente
shmid = shmget(key, size, IPC_CREAT | IPC_EXCL | S_IRUSR | S_IWUSR);
```

9.3.2 Agganciare un segmento di memoria condivisa (p. 28-30) `shmat()` - aggancia un segmento di memoria condivisa allo spazio di indirizzi del processo chiamante.

```
#include <sys/types.h>
#include <sys/shm.h>
void *shmat(int shmid, const void *shmaddr, int shmflg);
```

- **Ritorna:** l'indirizzo del segmento in caso di successo, -1 (castato a `void*`) in caso di errore.
- **errno in caso di errore:** EACCES, EINVAL.
- **shmid:** identificatore del segmento.
- **shmaddr:** indirizzo a cui agganciare il segmento. NULL: il sistema sceglie un indirizzo libero (raccomandato). Diverso da NULL: il kernel tenta di agganciare all'indirizzo specificato (deve essere allineato al confine di pagina, SHMLBA).
- **shmflg:** flag di controllo. SHM_RND: arrotonda `shmaddr` al multiplo di SHMLBA. SHM_RDONLY: aggancia il segmento in sola lettura. SHM_REMAP (Linux): rimpiazza una mappatura esistente.

Esempio di `shmat()` (p. 30):

```
// Aggancia la memoria condivisa nella modalita' lettura/scrittura
```

```

int *ptr_1 = (int *)shmat(shmid, NULL, 0);
// Aggancia la memoria condivisa nella modalita' sola lettura
int *ptr_2 = (int *)shmat(shmid, NULL, SHM_RDONLY);

// N.B. ptr_1 e ptr_2 sono diversi

// Scrive 10 interi nel segmento di memoria condivisa
for (int i = 0; i < 10; ++i) {
    ptr_1[i] = i;
}
// Legge 10 interi dal segmento di memoria condivisa
for (int i = 0; i < 10; ++i) {
    printf("integer: %d\n", ptr_2[i]);
}

```

9.3.3 Distaccare un segmento di memoria condivisa (p. 30-31) `shmdt()` - sgancia un segmento di memoria condivisa dal processo.

```

#include <sys/types.h>
#include <sys/shm.h>
int shmdt(const void *shmaddr);

```

- **Ritorna:** 0 in caso di successo, -1 in caso di errore (EINVAL).
- `shmaddr`: l'indirizzo restituito da `shmat()`.
- Aggiorna i campi di `shmid_ds`: `shm_dtime`, `shm_lpid`, `shm_nattch`.

Esempio di `shmdt()` (p. 31):

```

// Aggancia la memoria condivisa in modalita' lettura/scrittura
int *ptr_1 = (int *)shmat(shmid, NULL, 0);
if (ptr_1 == (void *)-1) {
    errExit("first shmat failed");
}

// Aggancia la memoria condivisa in modalita' sola lettura
int *ptr_2 = (int *)shmat(shmid, NULL, SHM_RDONLY);
if (ptr_2 == (void *)-1) {
    errExit("second shmat failed");
}

// Scollega i segmenti di memoria condivisa
if (shmdt(ptr_1) == -1 || shmdt(ptr_2) == -1) {
    errExit("shmdt failed");
}

```

9.3.4 Operazioni di controllo della memoria condivisa (p. 31-32) `shmctl()` - esegue operazioni di controllo su un segmento di memoria condivisa.

```

#include <sys/ipc.h>

```

```
#include <sys/shm.h>
int shmctl(int shmid, int cmd, struct shm_id *buf);
```

- **Ritorna:** 0 in caso di successo, -1 in caso di errore.
- **errno in caso di errore:** EACCES, EFAULT, EIDRM, EINVAL, ENOMEM, EOVERFLOW, EPERM.
- **cmd:** azione da eseguire (IPC_STAT, IPC_RMID, IPC_SET).

Esempio: Rimozione di un segmento di memoria condivisa (p. 32):

```
if (shmctl(shmid, IPC_RMID, NULL) == -1) {
    errExit("shmctl failed");
} else {
    printf("shared memory segment removed successfully\n");
}
```

4.2.4 9.4 Coda di messaggi (p. 32-39)

Le code di messaggi permettono a processi diversi di scambiarsi dati in forma di messaggi strutturati.

9.4.1 Creazione di una coda di messaggi (p. 32-33) `msgget()` - crea o ottiene una coda di messaggi.

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
int msgget(key_t key, int flag);
```

- **Ritorna:** l'identificatore (intero positivo) in caso di successo, -1 in caso di errore.
- **errno in caso di errore:** EACCES, EEXIST, EIDRM, ENOENT, ENOSPC.
- **key:** chiave associata (o IPC_PRIVATE).
- **flag:** permessi e opzioni (IPC_CREAT, IPC_EXCL).

Esempio di msgget() (p. 33):

```
int msqid;
key_t key = //... (si genera una chiave)

// A) Delega il problema di trovare una chiave unica al kernel
msqid = msgget(IPC_PRIVATE, S_IRUSR | S_IWUSR);

// B) Crea una coda con l'identificatore key, se non e' gia' esistente
msqid = msgget(key, IPC_CREAT | S_IRUSR | S_IWUSR);

// C) Crea una coda con l'identificatore key, ma fallisce se e' gia' esistente
msqid = msgget(key, IPC_CREAT | IPC_EXCL | S_IRUSR | S_IWUSR);
```

9.4.2 Struttura dei messaggi (p. 33-34) Una coda di messaggi è una lista di messaggi. Ogni coda ha associata una struttura `msqid_ds`. La struttura di un messaggio `msgbuf`:

```

struct msgbuf {
    long mtype;    // Deve essere > 0
    char mtext[LENGTH]; // Messaggio
};

```

9.4.3 Invio di messaggi (p. 34-36) `msgsnd()` - invia un messaggio su una coda.

```

#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
int msgsnd(int msqid, struct msgbuf *msgp, size_t msgsz, int msgflg);

```

- **Ritorna:** 0 in caso di successo, -1 in caso di errore.
- **errno in caso di errore:** EACCES, EAGAIN, EIDRM, EINVAL.
- **msqid:** identificatore della coda.
- **msgp:** puntatore alla struttura del messaggio.
- **msgsz:** dimensione del testo del messaggio (`mtext`). Può essere 0.
- **msgflg:** flag di controllo (es. `IPC_NOWAIT`).

Esempio di `msgsnd()` (p. 35):

```

// Struttura del messaggio
struct mymsg {
    long mtype;
    char mtext[100]; /* Messaggio */
} m;

// Il messaggio ha come tipo 1
m.mtype = 1;

// Il messaggio contiene la seguente stringa
char *text = "Ciao mondo!";
memcpy(m.mtext, text, strlen(text) + 1);

// La dimensione di m e' solo la dimensione dell'attributo mtext
size_t mSize = sizeof(struct mymsg) - sizeof(long);

// Invio del messaggio nella coda
if (msgsnd(msqid, &m, mSize, 0) == -1) {
    errExit("msgsnd failed");
}

```

Esempio con corpo di due interi (p. 35):

```

// Struttura del messaggio
struct mymsg {
    long mtype;
    int num1, num2; /* Due interi come corpo */
}

```

```

} m;

m.mtype = 2;
m.num1 = 34;
m.num2 = 43;
size_t mSize = sizeof(struct mymsg) - sizeof(long);

if (msgsnd(msqid, &m, mSize, 0) == -1) {
    errExit("msgsnd failed");
}

```

Esempio con solo il tipo (p. 36):

```

// Struttura del messaggio
struct mymsg {
    long mtype;
} m;

m.mtype = 3;
size_t mSize = sizeof(struct mymsg) - sizeof(long); // 0

if (msgsnd(msqid, &m, mSize, IPC_NOWAIT) == -1) {
    if (errno == EAGAIN) {
        printf("The queue was full!\n");
    } else {
        errExit("msgsnd failed");
    }
}

```

9.4.4 Ricezione di messaggi (p. 36-38) msgrcv() - legge un messaggio da una coda.

```

#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
size_t msgrcv(int msqid, struct msgbuf *msgp, size_t msgsz, long msgtyp, int msgflg);

```

- **Ritorna:** il numero di byte letti in caso di successo, -1 in caso di errore.
- **errno in caso di errore:** E2BIG, EACCES, EIDRM, EINTR, EINVAL.
- **msqid:** identificatore della coda.
- **msgp:** puntatore alla struttura dove scrivere il messaggio.
- **msgsz:** lunghezza massima del testo del messaggio. Se il messaggio è più lungo e MSG_NOERROR non è impostato, la funzione fallisce con E2BIG.
- **msgtyp:** specifica il tipo di messaggio da leggere. = 0: legge il primo messaggio in coda. > 0: legge il primo messaggio con tipo uguale a msgtyp. < 0: legge il primo messaggio con il tipo più basso tra quelli con tipo < |msgtyp|.
- **msgflg:** flag di controllo (es. IPC_NOWAIT, MSG_NOERROR).

Esempio di msgrcv() (p. 37):

```
// Struttura del messaggio
struct mymsg {
    long mtype;
    char mtext[100];
} m;

size_t mSize = sizeof(struct mymsg) - sizeof(long);

// Si attende per un messaggio avente tipo uguale a 1
if (msgrcv(msqid, &m, mSize, 1, 0) == -1) {
    errExit("msgrcv failed");
}
```

Esempio con MSG_NOERROR (p. 37):

```
struct mymsg {
    long mtype;
    char mtext[100];
} m;

size_t mSize = sizeof(char) * 50;

// Si attende per un messaggio avente tipo uguale a 1,
// ma vengono copiati solo i primi 50 byte
if (msgrcv(msqid, &m, mSize, 1, MSG_NOERROR) == -1) {
    errExit("msgrcv failed");
}
```

Esempio di polling con IPC_NOWAIT (p. 37-38):

```
struct mymsg {
    long mtype;
} m;

// Nella modalita' polling, si prova a ottenere
// un messaggio a ogni secondo che passa
while (1) {
    sleep(1);
    // Esecuzione di un msgrcv non-bloccante
    if (msgrcv(msqid, &m, 0, 3, IPC_NOWAIT) == -1) {
        if (errno == ENOMSG) {
            printf("No message with type 3 in the queue\n");
        } else {
            errExit("msgrcv failed");
        }
    } else {
        printf("I found a message with type 3\n");
    }
}
```



```
}
```

9.4.5 Operazioni di controllo (p. 38-39) `msgctl()` - esegue operazioni di controllo su una coda di messaggi.

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
int msgctl(int msqid, int cmd, struct msqid_ds *buf);
```

- **Ritorna:** 0 in caso di successo, -1 in caso di errore.
- **errno in caso di errore:** EACCES, EIDRM, EPERM.
- **cmd:** azione da eseguire (IPC_STAT, IPC_RMID, IPC_SET).

Esempio: Rimozione di una coda (p. 39):

```
if (msgctl(msqid, IPC_RMID, NULL) == -1) {
    errExit("msgctl failed");
} else {
    printf("message queue removed successfully\n");
}
```

Esempio: Cambiare i limiti superiori della coda (p. 39):

```
struct msqid_ds ds;

// Ottiene la struttura dati della coda di messaggi
if (msgctl(msqid, IPC_STAT, &ds) == -1) {
    errExit("msgctl");
}

// Cambia i limiti superiori del numero di byte nel campo
// mtext di tutti i messaggi della coda di messaggi a 1 Kbyte
ds.msg_qbytes = 1024;

// Carica la struttura dati associata nel kernel
if (msgctl(msqid, IPC_SET, &ds) == -1) {
    errExit("msgctl");
}
```

4.2.5 9.5 PIPE (p. 39-42)

Le pipe sono il meccanismo IPC più antico. Sono un **buffer circolare in memoria** a cui si accede tramite una coppia di file descriptor: `fd[0]`: estremità di lettura; `fd[1]`: estremità di scrittura.

Proprietà (p. 39-40):

- **Unidirezionali:** i dati viaggiano in una sola direzione.
- **Sequenziali:** i byte vengono letti nello stesso ordine in cui sono scritti.
- **Nessun concetto di messaggio:** si leggono blocchi di byte di qualsiasi dimensione.

- **Lettura da pipe vuota:** blocca il lettore.
- **Scrittura su pipe chiusa in lettura:** genera SIGPIPE.
- **Scrittura:** è bloccante fino a quando non c'è spazio sufficiente per completare l'operazione atomicamente.

9.5.1 Creazione di una PIPE (p. 40-42) `pipe()` - crea una pipe.

```
#include <unistd.h>
int pipe(int filedes[2]);
```

Ritorna: 0 in caso di successo, -1 in caso di errore. `filedes[0]` è il descrittore di lettura, `filedes[1]` quello di scrittura.

Esempio: Comunicazione padre-figlio con pipe (p. 41-42):

```
int fd[2];

// Si verifica se PIPE ha successo
if (pipe(fd) == -1) {
    errExit("PIPE");
}

// Si crea un processo figlio
switch(fork()) {
    case -1:
        errExit("fork");
    case 0: // Figlio
        // ... il figlio legge dalla PIPE
        break;
    default: // Padre
        // ... il padre scrive alla PIPE
        break;
}
```

Codice del processo figlio (p. 41):

```
char buf[SIZE];
ssize_t nBys;

// Chiude l'estremità di scrittura inutilizzata
if (close(fd[1]) == -1) {
    errExit("close - child");
}

// Si legge dalla PIPE
nBys = read(fd[0], buf, SIZE);

// 0: end-of-file, -1: fallisce
if (nBys > 0) {
    buf[nBys] = '\0';
```

```

    printf("%s\n", buf);
}

// Chiude l'estremita' di lettura della PIPE
if (close(fd[0]) == -1) {
    errExit("close - child");
}

```

Codice del processo padre (p. 41-42):

```

char buf[] = "Ciao Mondo\n";
ssize_t nBys;

// Chiude l'estremita' di lettura inutilizzata
if (close(fd[0]) == -1) {
    errExit("close - parent");
}

// Si scrive alla PIPE
nBys = write(fd[1], buf, strlen(buf));

// Si verifica se la scrittura ha avuto successo
if (nBys != (ssize_t)strlen(buf)) {
    errExit("write - parent");
}

// Si chiude l'estremita' di scrittura della PIPE
if (close(fd[1]) == -1) {
    errExit("close - parent");
}

```

4.2.6 9.6 FIFO (p. 42-43)

Le FIFO (named pipes) hanno le stesse caratteristiche delle pipe, ma risiedono nel filesystem come file speciali. Permettono la comunicazione tra processi non imparentati.

9.6.1 Creazione di una FIFO (p. 42) `mkfifo()` - crea una FIFO.

```

#include <sys/types.h>
#include <sys/stat.h>
int mkfifo(const char *pathname, mode_t mode);

```

9.6.2 Apertura di una FIFO (p. 43) L'apertura avviene con `open()`. `open(pathname, O_RDONLY)` si blocca fino a quando un altro processo apre la FIFO in scrittura. `open(pathname, O_WRONLY)` si blocca fino a quando un altro processo apre la FIFO in lettura.

Esempio: Comunicazione mediante FIFO (p. 43):

Destinatario (p. 43):

```

char *fname = "/tmp/myfifo";

```

```
int res = mkfifo(fname, S_IRUSR | S_IWUSR);
```

```
// Apertura in modalita' di sola lettura  
int fd = open(fname, O_RDONLY);
```

```
// Lettura dei byte da un fifo  
char buffer[LEN];  
read(fd, buffer, LEN);
```

```
// Stampa del buffer in output  
printf("%s\n", buffer);
```

```
// Chiusura della FIFO  
close(fd);
```

```
// Rimozione della FIFO  
unlink(fname);
```

Mittente (p. 43):

```
char *fname = "/tmp/myfifo";
```

```
// Apertura in modalita' di sola scrittura  
int fd = open(fname, O_WRONLY);
```

```
// Lettura di una stringa (senza spazi)  
char buffer[LEN];  
printf("Give me a string: ");  
scanf("%s", buffer);
```

```
// Si scrive la stringa all'interno della fifo  
write(fd, buffer, strlen(buffer));
```

```
// Chiusura della FIFO  
close(fd);
```